上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU

# UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space


**Yilong Zhao, Ph.D Student**

**Shanghai Jiao Tong University**

**Advisor: Prof. Li Jiang**


March 23, 2026

饮 水 思 源 • 爱 国 荣 校

# Outline

## UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

in ISCA'24

**Yilong Zhao**, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University and Huawei*

## PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

in ASPLOS'25

**Yilong Zhao**, Mingyu Gao, Huanchen Zhang, Fangxin Liu,* Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University*

Wednesday, **March 25 8:30 – 10:35 AM**
**OHIO & MONOGAHELA**, Session 4D

# Outline

## UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

in ISCA'24

**Yilong Zhao,** Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University and Huawei*

## PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

in ASPLOS'25

**Yilong Zhao**, Mingyu Gao, Huanchen Zhang, Fangxin Liu,* Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University*

Wednesday, **March 25 8:30 – 10:35 AM**
**OHIO & MONOGAHELA**, Session 4D

# UM-PIM [ISCA'24]

- **Problem：** data layout conflict -- **CPU memory interleaving vs. PIM locality**
- **Drawback of current solution:**
  - Isolated memory space (CPU space + PIM space)→ Additional data movement
- **Insight:** One **<u>uniform memory space</u>**, with CPU & PIM co-exist in it
- **Challenges → Solutions：**

| | | |
|---|---|---|
| 1 Co-existence of CPU/PIM Pages | → | Chunk-based memory management (SW) |
| 2 CPU/PIM Page Layouts | → | Dynamic Memory Mapping + Data Re-layout Module (HW on DRAM side) |

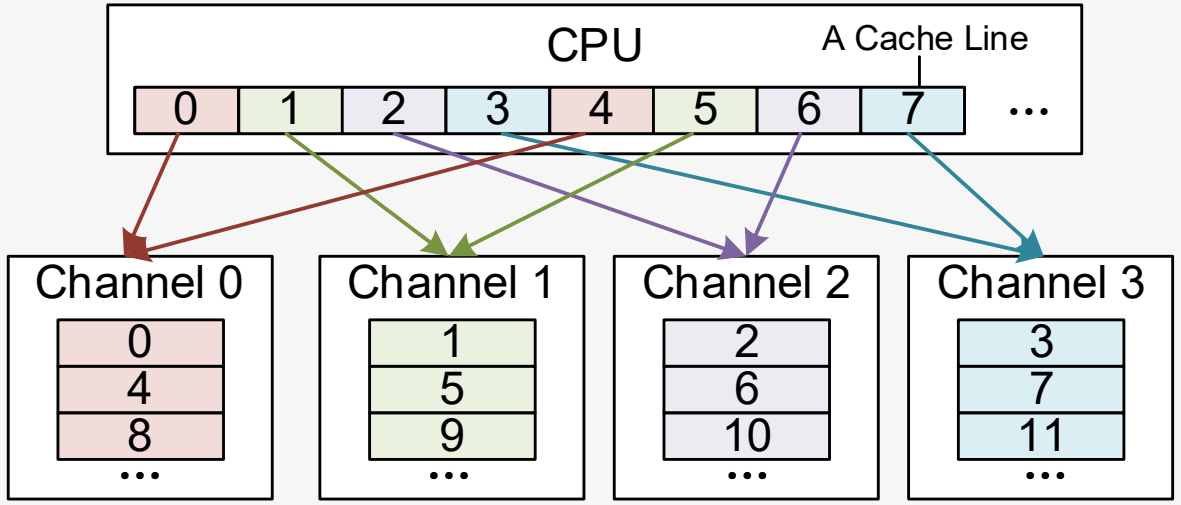
- **Evaluation：** Gem5+Ramulator2 simulation | 8-Channel DDR4 | PRIM benchmark
- **Results：** 1.96× peak PIM speedup (to baseline with isolated memory space) | < 0.1% CPU degradation

# Background: Memory Interleaving

**Memory Interleaving:** increase CPU bandwidth

Adjacent data block stored in different modules,
→ Can be accessed in parallel/with pipeline

An example of interleaving (on channel level):

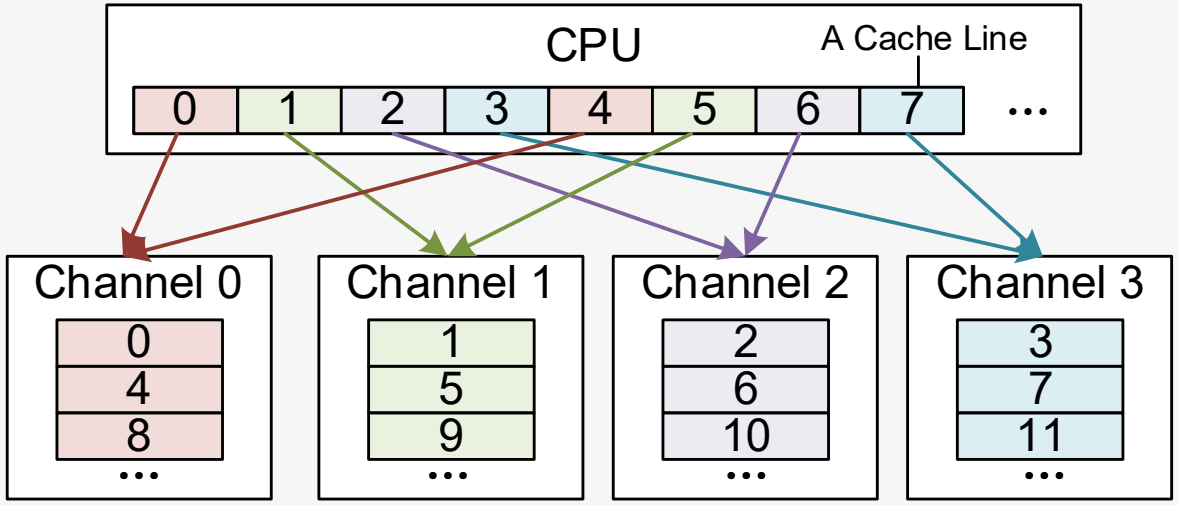


# Background: Memory Interleaving

**Memory Interleaving:** increase CPU bandwidth

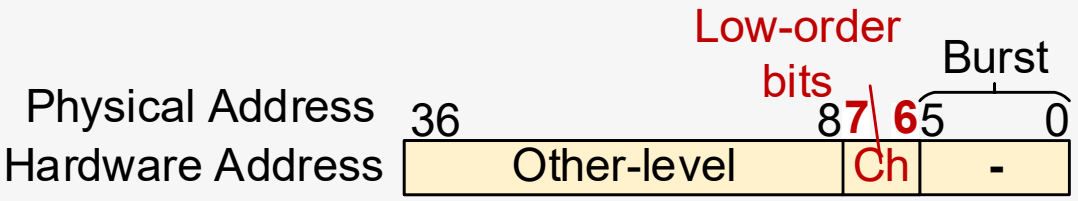
Adjacent data block stored in different modules,
→ Can be accessed in parallel/with pipeline

An example of interleaving (on channel level):




Implement channel interleaving through **address mapping**:

Map the low-order bits (bit 6,7) to channel




Physical Address

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | - | - |
| 1 | 0 | 1 | - | - |
| 2 | 0 | 2 | - | - |
| 3 | 0 | 3 | - | - |
| 4 | 1 | 0 | - | - |
| 5 | 1 | 1 | - | - |
| 6 | 1 | 2 | - | - |
| 7 | 1 | 3 | - | - |

...

## Extend memory interleaving to all DRAM levels:

Map address according to **switch overhead**

Map **lower/high**-order bits to **low/high**-switch-overhead levels

High

Row (>100 cycles)

Rank (10-18 cycles)

Bank (8-16 cycles)

Column (~8 cycles)

Channel (~8 cycles)

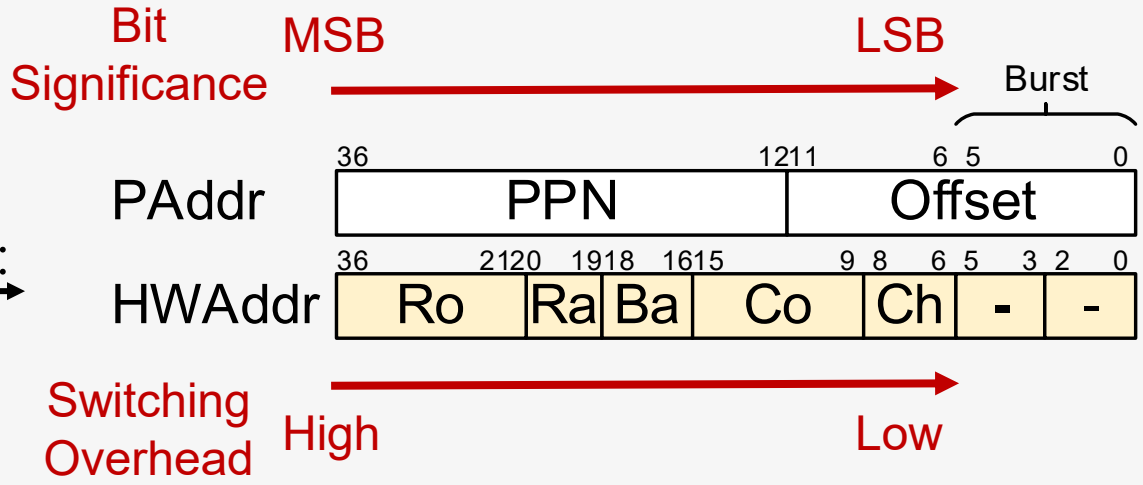
Device (Parallel)

Low

Switching Overhead

(Data source: JEDEC)

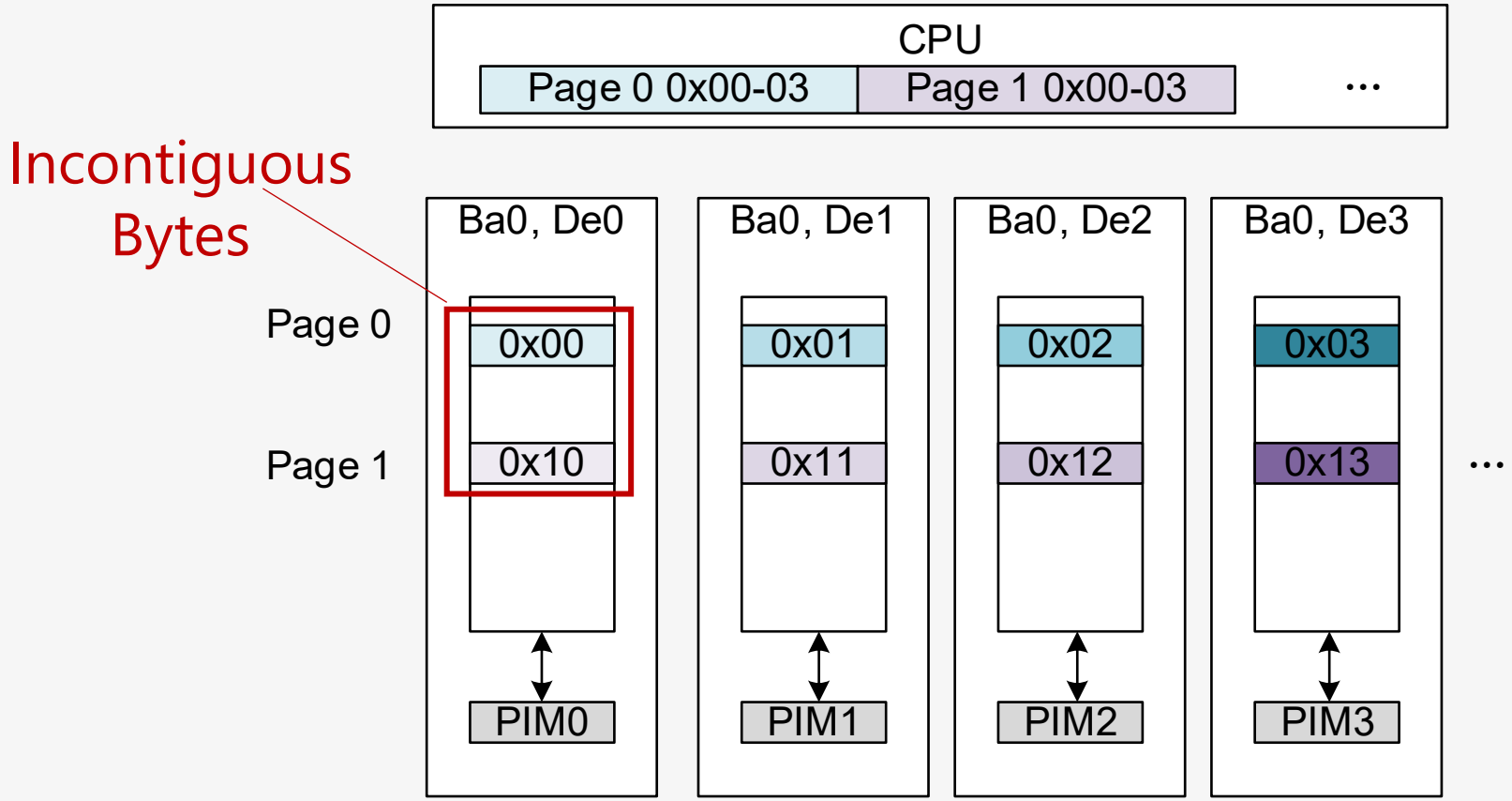
Address Mapping:




(PAddr: Physical Address
HWAddr: Hardware Address)

# Limitation: Poor Locality of PIM Workload (1)

1 Contiguous virtual pages are mapped to non-contiguous physical frames.
2 Physical pages are interleaved to different sub-modules
→ **PIM only visible to short contiguous data block**



# Limitation: Poor Locality of PIM Workload (2)

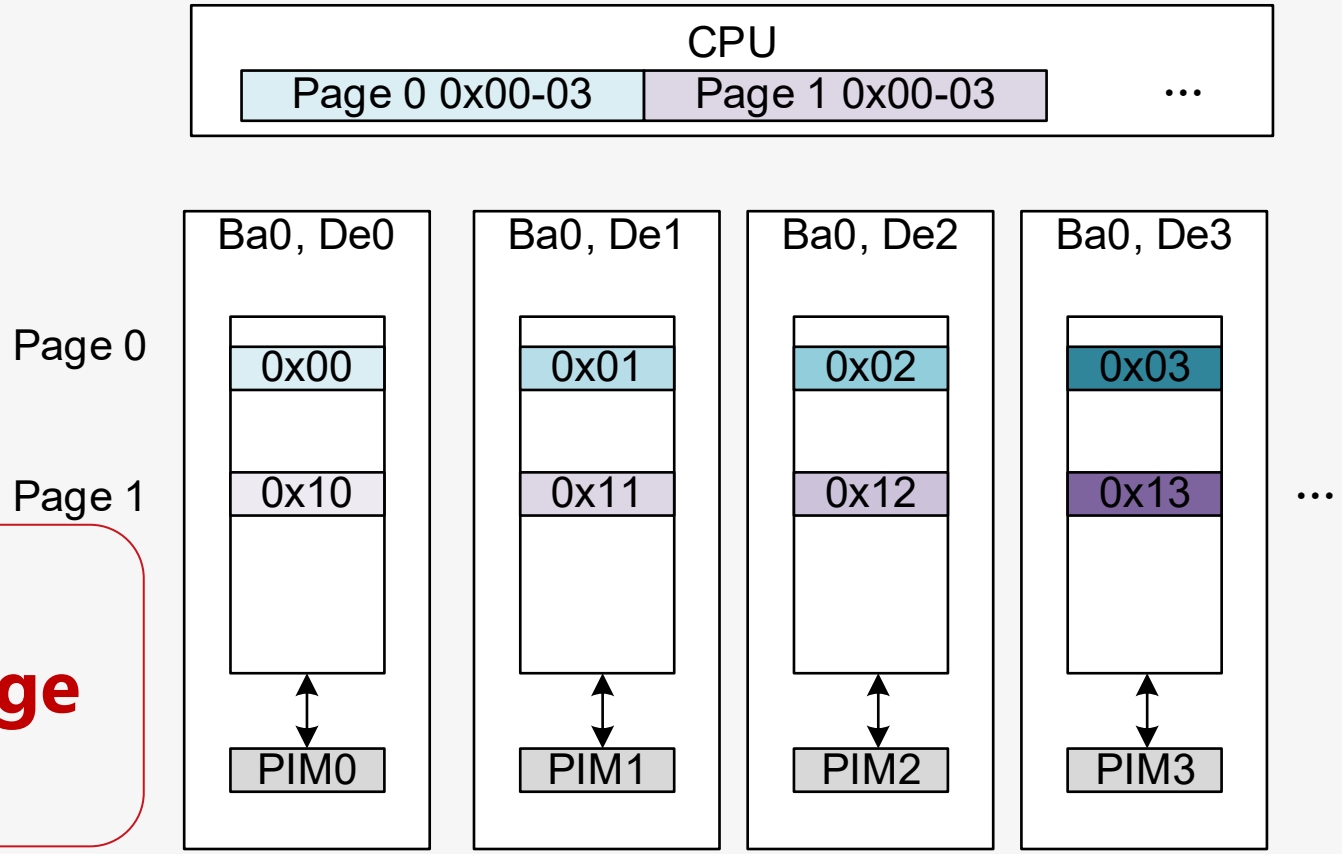
Inefficiency caused by poor locality: $n$ pages needs $n$ offloads

**Offload:**
Transfer start address, context…

~50us/ offload

**Task:**
Process 1-byte data

~30ns/ task



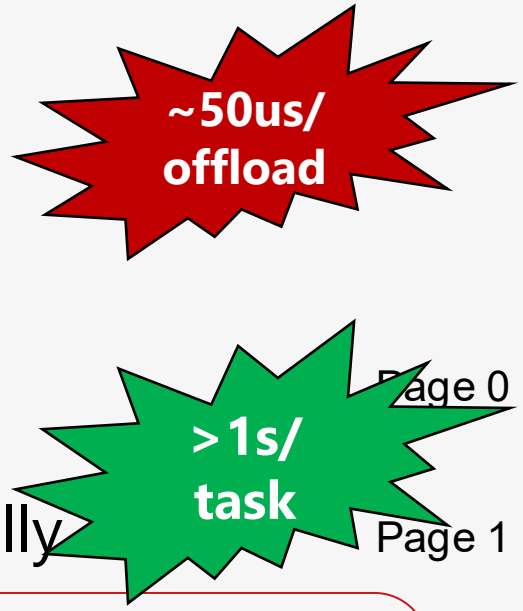

**→ Offload tiny PIM workloads many times, each time incurs large overhead ×**

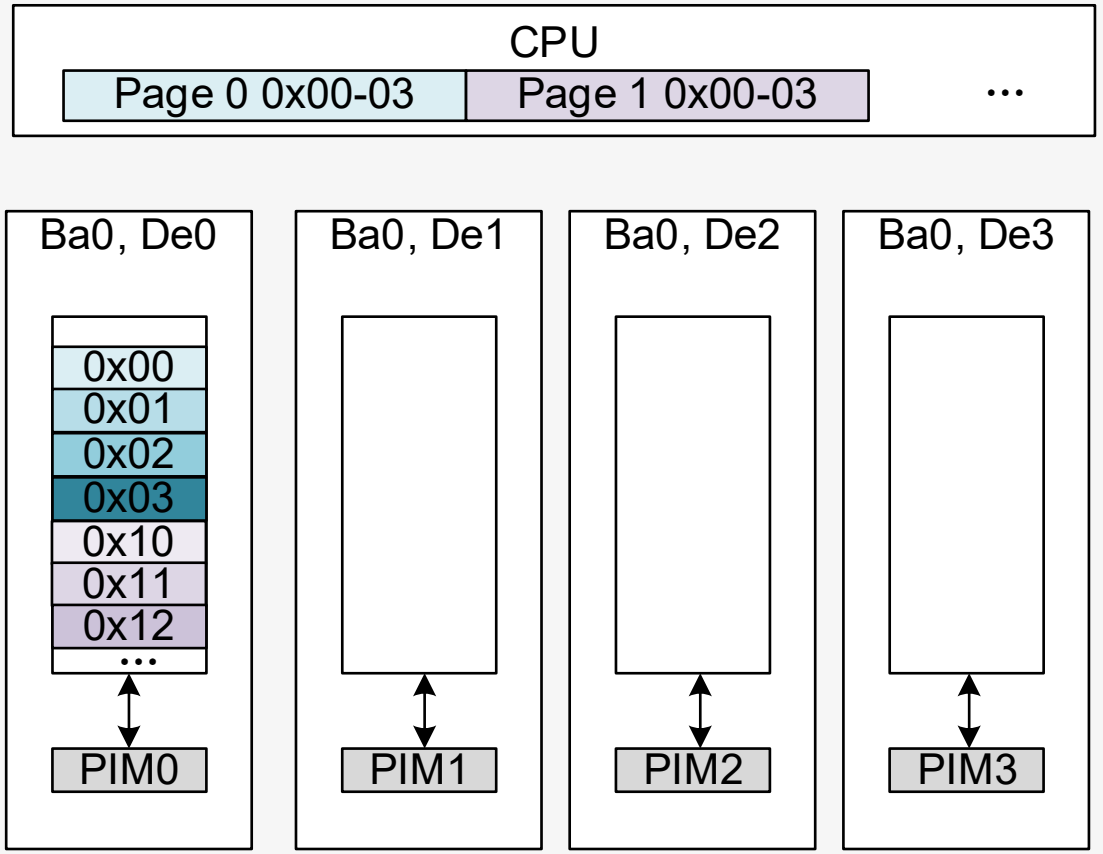
# An Ideal Data Layout for PIM

If data is non-interleaved, the same task only need to offload once!

**Offload:**
Transfer start address, context…

**Task:**
PIM automatically process subsequent data sequentially

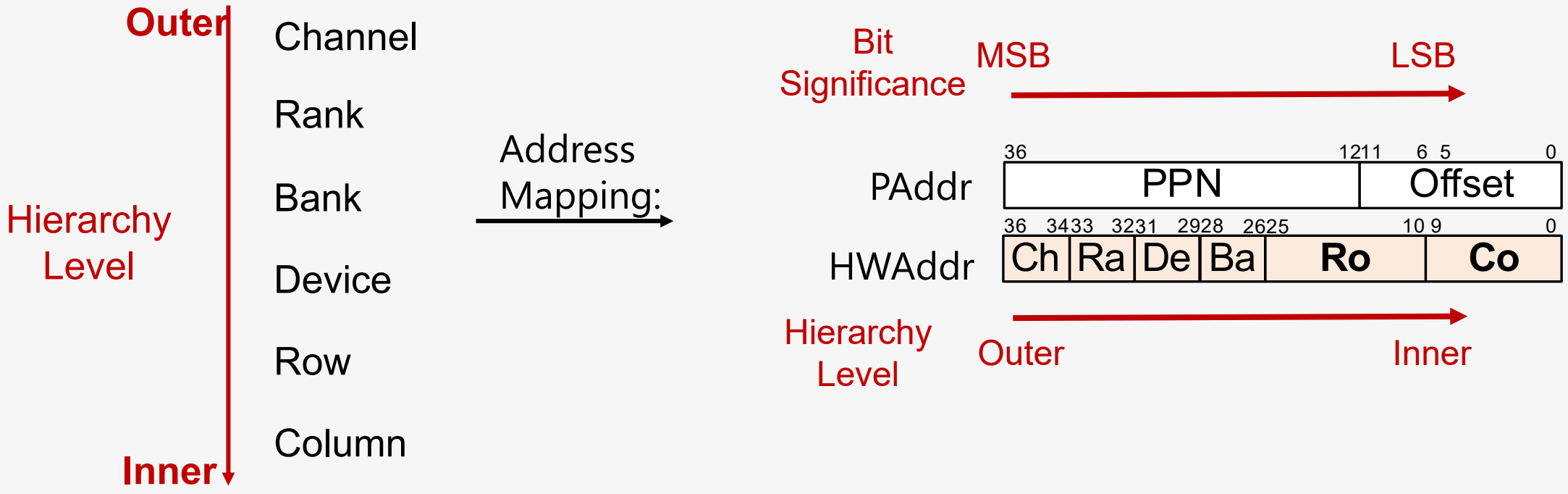


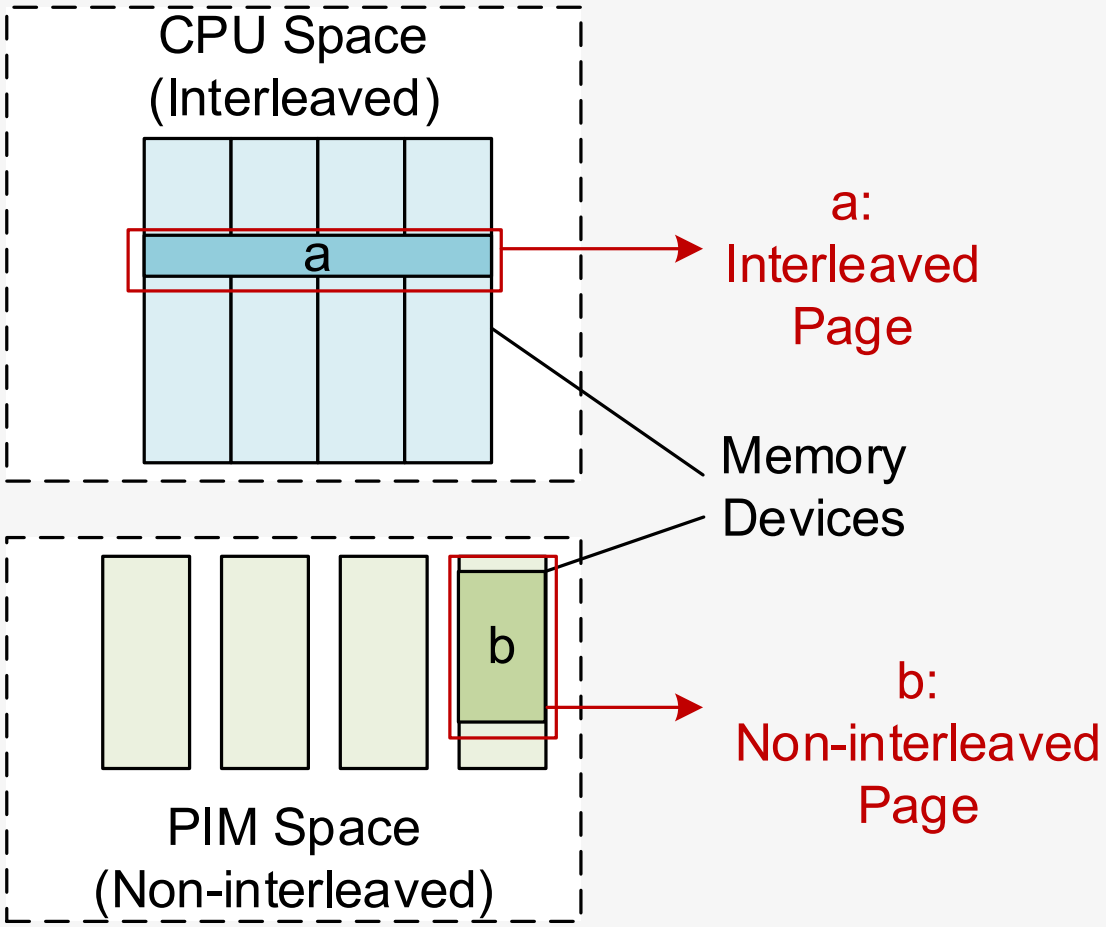

**→ One offload followed by a long task, amortizing offload overhead √**

## Address mapping of this ideal data layout :

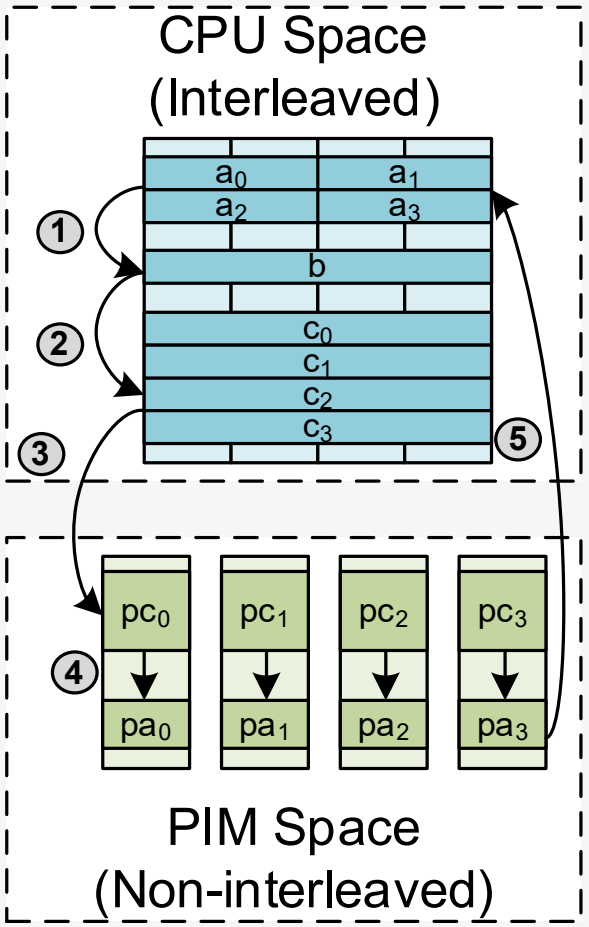
Mapping the address bits by memory hierarchy



# Existing Solutions 1 – Isolated Memory Space



# Computation under Isolated Memory Space:




Example: MLP

***Program:***

1 2 **(CPU Compute)**

*b=reduce(a)*

*c=softmax(b)*

3 **(CPU-PIM Move)**

*$buf_0$=reshape($c_0$)*

*memcpy($pc_0$, $buf_0$)*

...

4 **(PIM Compute)**

*pa=gemm(pc)*

5 **(PIM-CPU Move)**

*$buf_0$=reshape($pa_0$)*

*memcpy($a_0$, $buf_0$)*

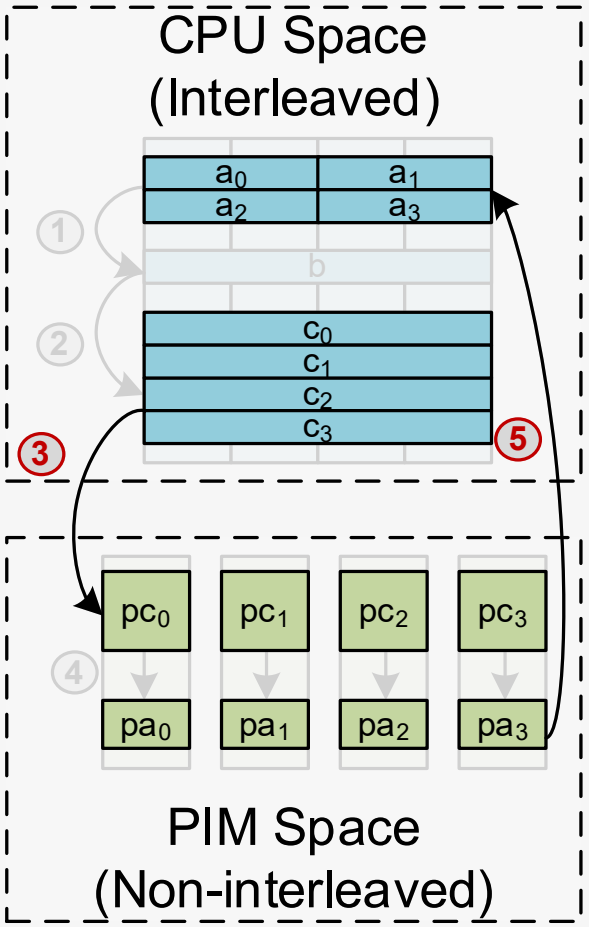
...

Offload

**4 Stages**

# Existing Solutions 1 – Isolated Memory Space

## Computation under Isolated Memory Space:




Example: MLP

***Program:***

①② (CPU Compute)

*b=reduce(a)*

*c=softmax(b)*

③ **(CPU-PIM Move)**

*$buf_0$=reshape($c_0$)*

*memcpy($pc_0$, $buf_0$)*

...

④ (PIM Compute)

*pa=gemm(pc)*

⑤ **(PIM-CPU Move)**

*$buf_0$=reshape($pa_0$)*

*memcpy($a_0$, $buf_0$)*

...

Offload

**Data Re-layout:**

Interleaved Page $C0$

↓

Non-interleaved Page $PC0$

**Memory Copy:**

CPU Space → PIM Space

Data Re-layout

Memory Copy

# Existing Solutions 1 – Isolated Memory Space

## Computation under Isolated Memory Space:




Example: MLP

***Program:***

①② (CPU Compute)

*b=reduce(a)*

*c=softmax(b)*

③ **(CPU-PIM Move)**

*$buf_0$=reshape($c_0$)*

*memcpy($pc_0$, $buf_0$)*

...

④ (PIM Compute)

*pa=gemm(pc)*

⑤ **(PIM-CPU Move)**

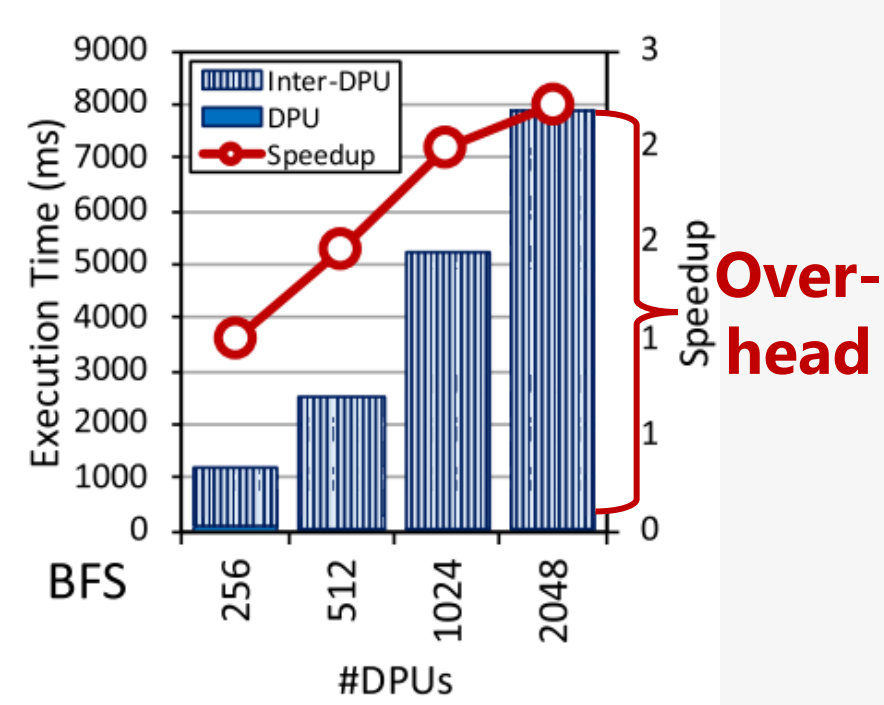
*$buf_0$=reshape($pa_0$)*

*memcpy($a_0$, $buf_0$)*

...

Offload

3, 5: Additional data copy and re-layout!




Gómez-Luna et.al. 2022

# Existing Solutions 2 – Half interleaving

**Switch off Interleaving of Channel & Rank**

Reduce re-layout overhead, but damage CPU bandwidth

Data re-layout overhead still accounts for 70% of data transfer time

**Re-layout Overhead:**

Solution 1: Re-layout Channel, Rank, Bank, Device

↓ Reduce re-layout overhead

Solution 2: Re-layout Bank, Device ~~(Channel, Rank)~~

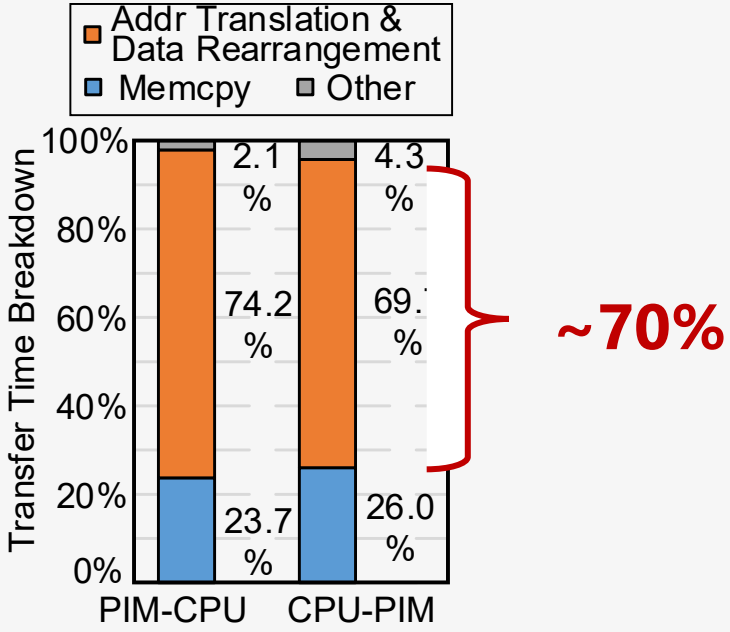
***Trade-Off***

**CPU Bandwidth:**

Solution 1: Mapping is Row-Rank-Bank-Column-Channel

↓ Worse CPU bandwidth

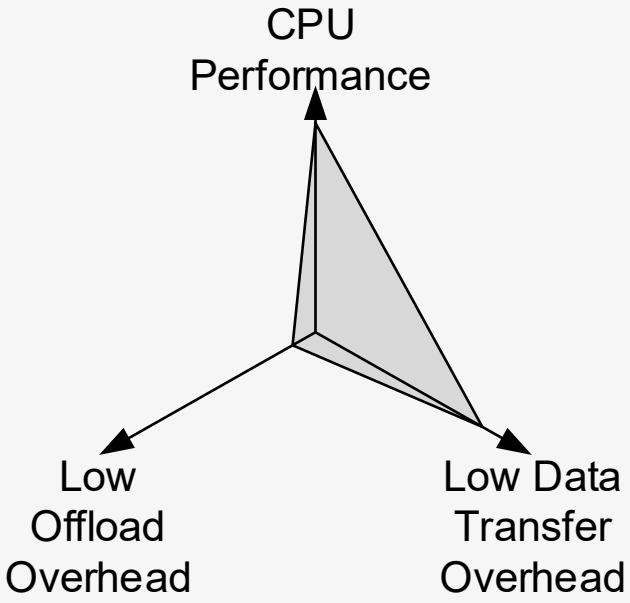
Solution 2: Mapping is Channel-Rank-Row-Bank-Column




Profiling the data transfer overhead on UPMEM DRAM-PIM

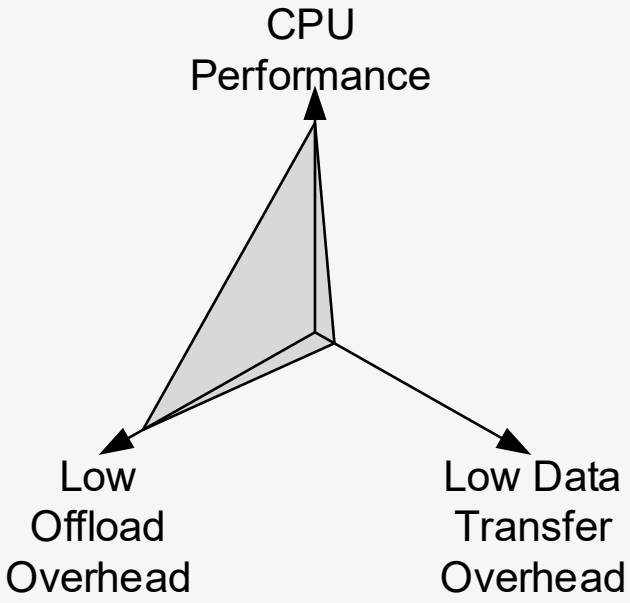
# Summary of Current Solutions: An Impossible Trinity

- Unified momory needs a uniform data layout
- A impossible triangle **without hardware modification**



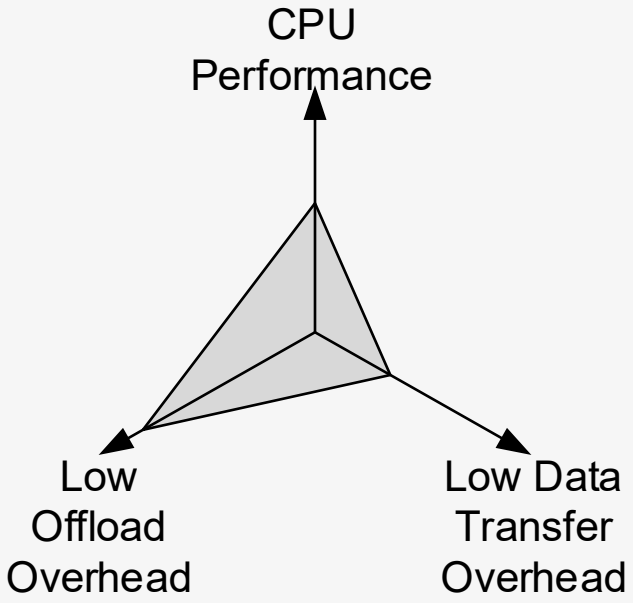

**(Origin)**

Uniform Layout
Fine-grained Offload




**Solution 1**

Isolated Memory Space
Software Re-layout
MetaPNM,PIM-HBM

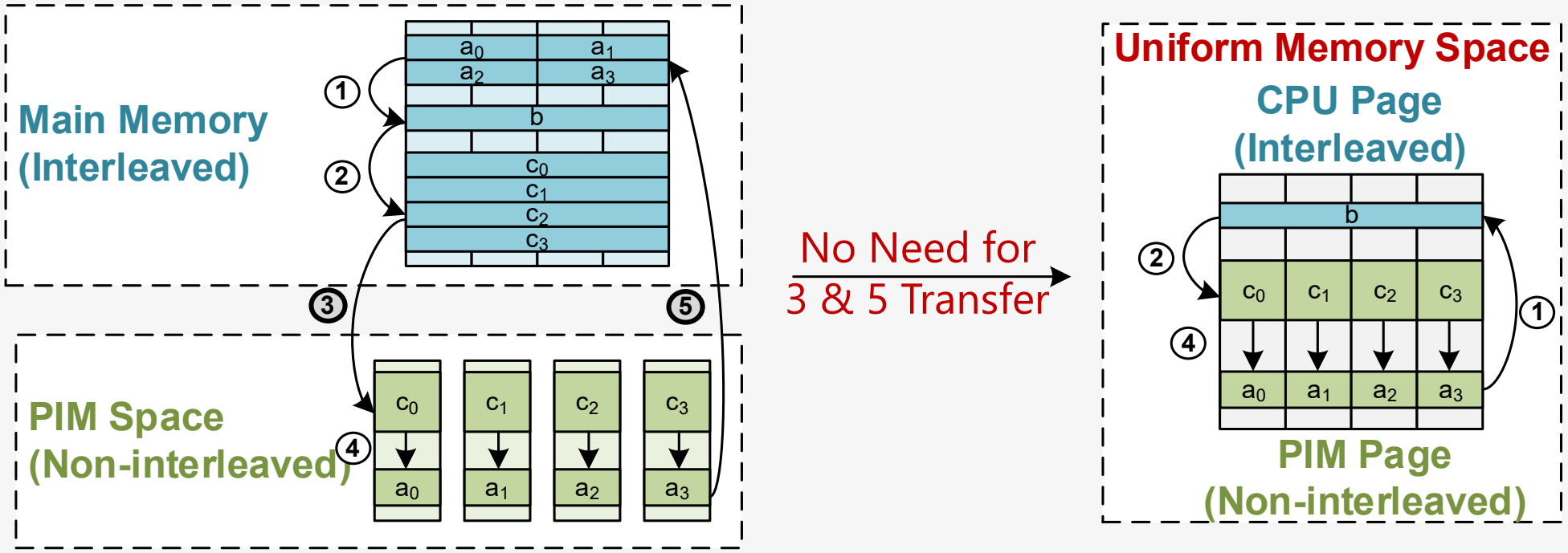



**Solution 2**

Isolated Memory Space
Switch off Interleaving
AxDIMM, UPMEM

# UM-PIM Contribution

- **Key insight**: A **<u>Uniform Memory Space</u>** with CPU & PIM Pages (different layout) co-existing in the space
  - No Data Transfer Overhead
  - Satisfy CPU and PIM's different data-layouts




Current PIM System

UM-PIM

# UM-PIM Challenges

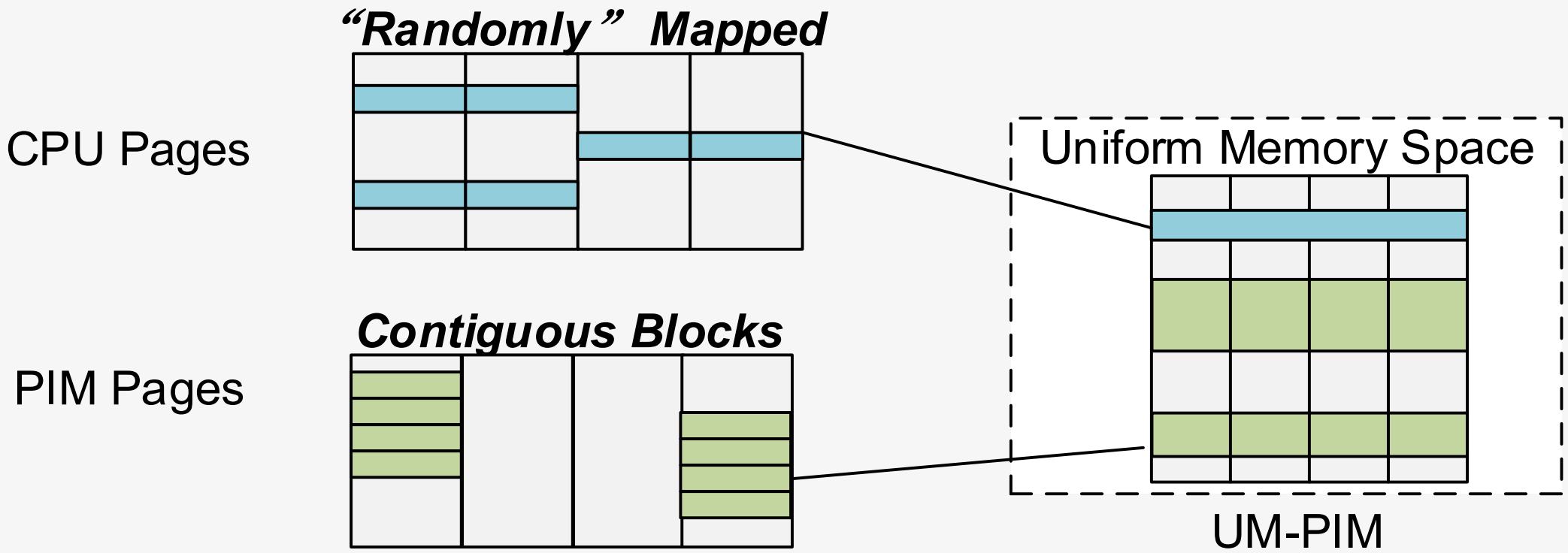
- **Key insight**: A **Uniform Memory Space** with CPU & PIM Pages (different layout) co-existing in the space

**Challenge 1: Co-existing**

**Challenge 2: Different Layout**

# UM-PIM Motivation: A Uniform Memory Space

- **Challenge 1: co-existence of CPU & PIM's pages under a single VM space**

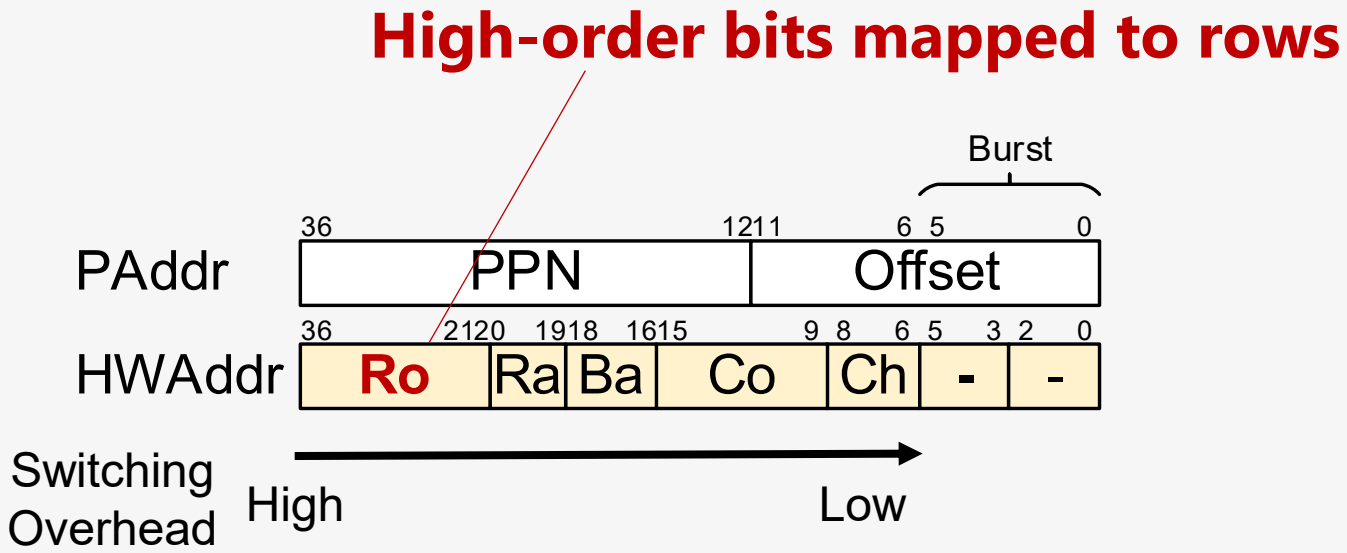



**How to ensure two kinds of Pages co-existing in the space?**
**Solution A.** Chunk-based Management

# A. Memory Management – CPU Malloc ①

- CPU Pages: keep current management strategy
- PIM Pages: Chunk-based Management

**Address Mapping:**
**Interleaved**

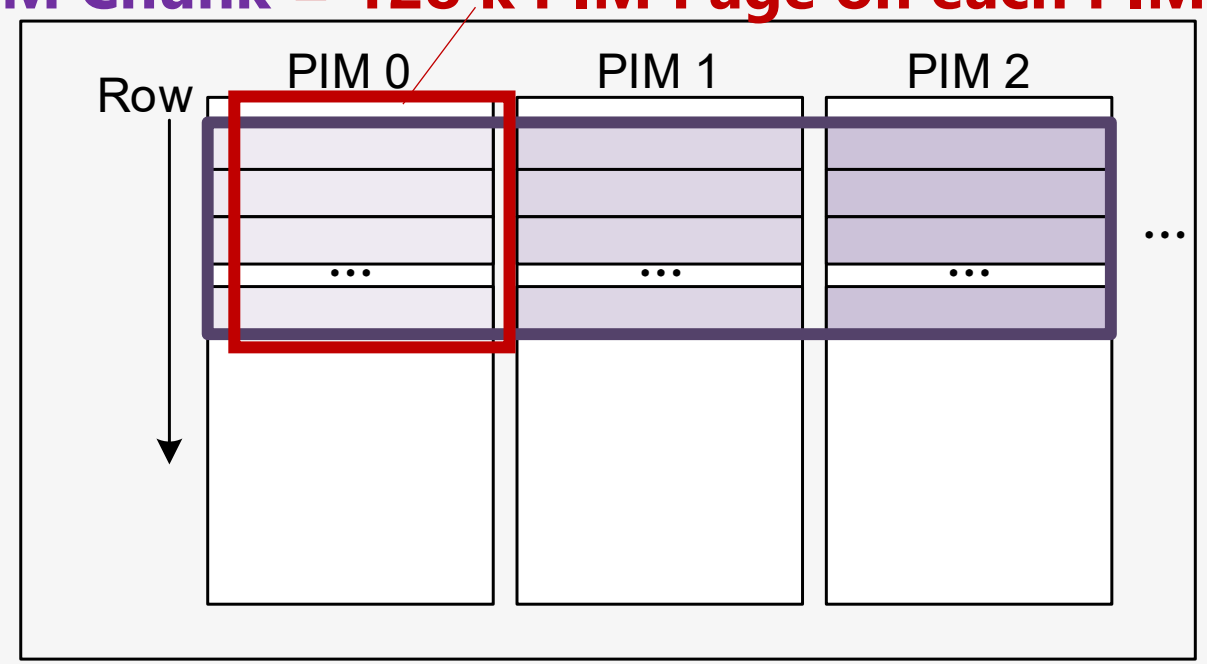


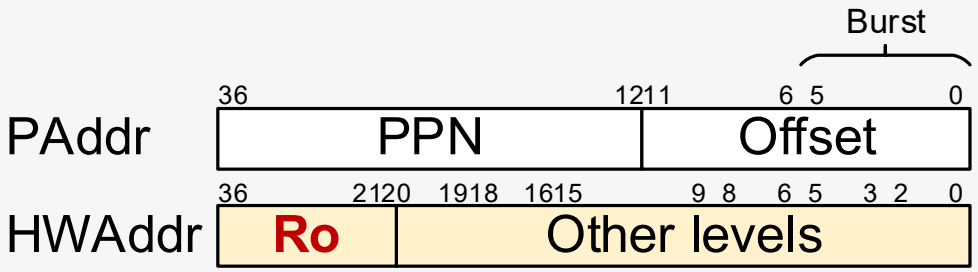
# A. Memory Management – CPU Allocate PIM Page ①

System Config:
8 Channels * 4 Ranks *
16 Banks * 8 Devices

- PIM Pages: Chunk-based Management
  - Allocate a chunk through **transparent huge page (THP)** (e.g., 256M)
  - Chunk evenly distributed on every PIMs (with *row* mapped to higher-order bits)

**256M Chunk = 128 k PIM Page on each PIM**



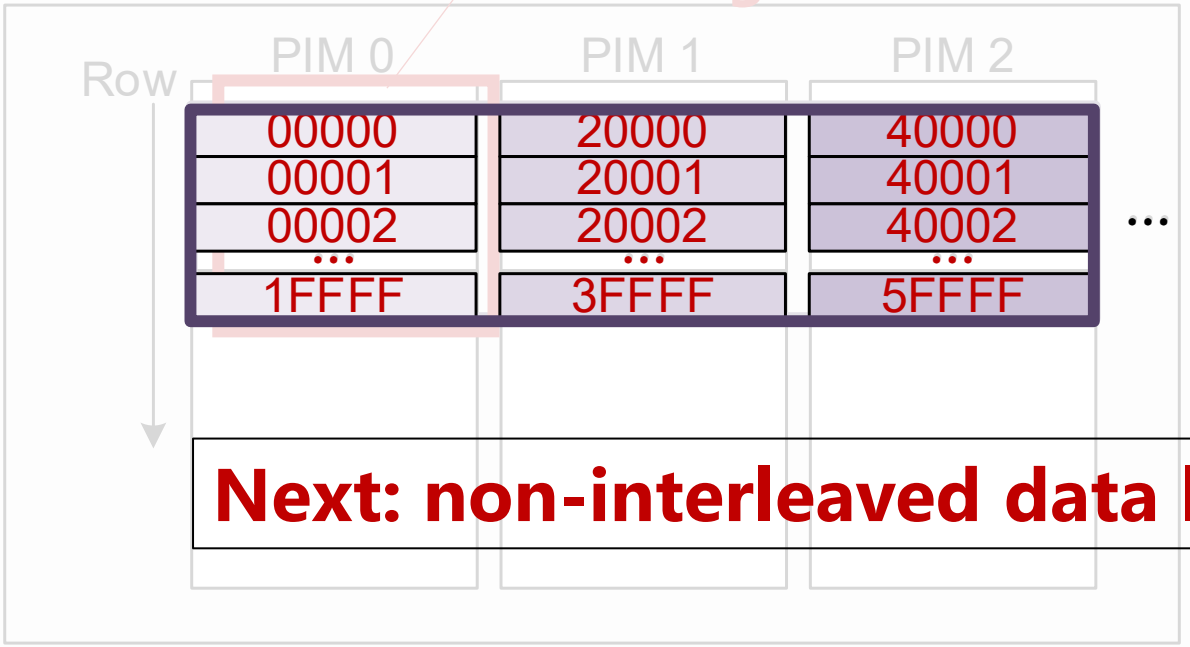
# A. Memory Management – CPU Allocate PIM Page ①

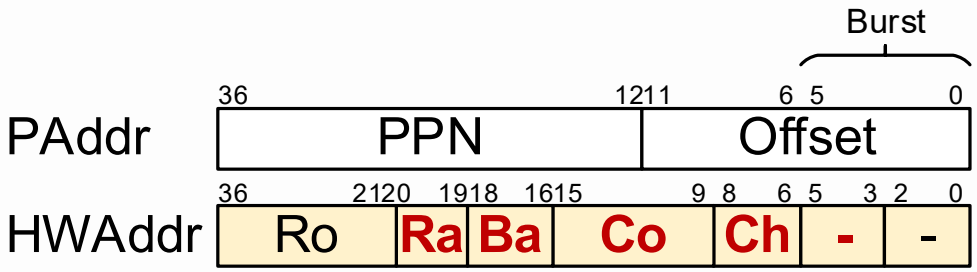
System Config:
8 Channels * 4 Ranks *
16 Banks * 8 Devices

PIM Pages: Huge-page-based Management

- Allocate a chunk through **transparent huge page (THP)** (e.g., 256M)
- Chunk evenly distributed on every PIMs (Row is mapped to higher-order bits)

**256M Chunk = 128 k PIM Page on each PIM**

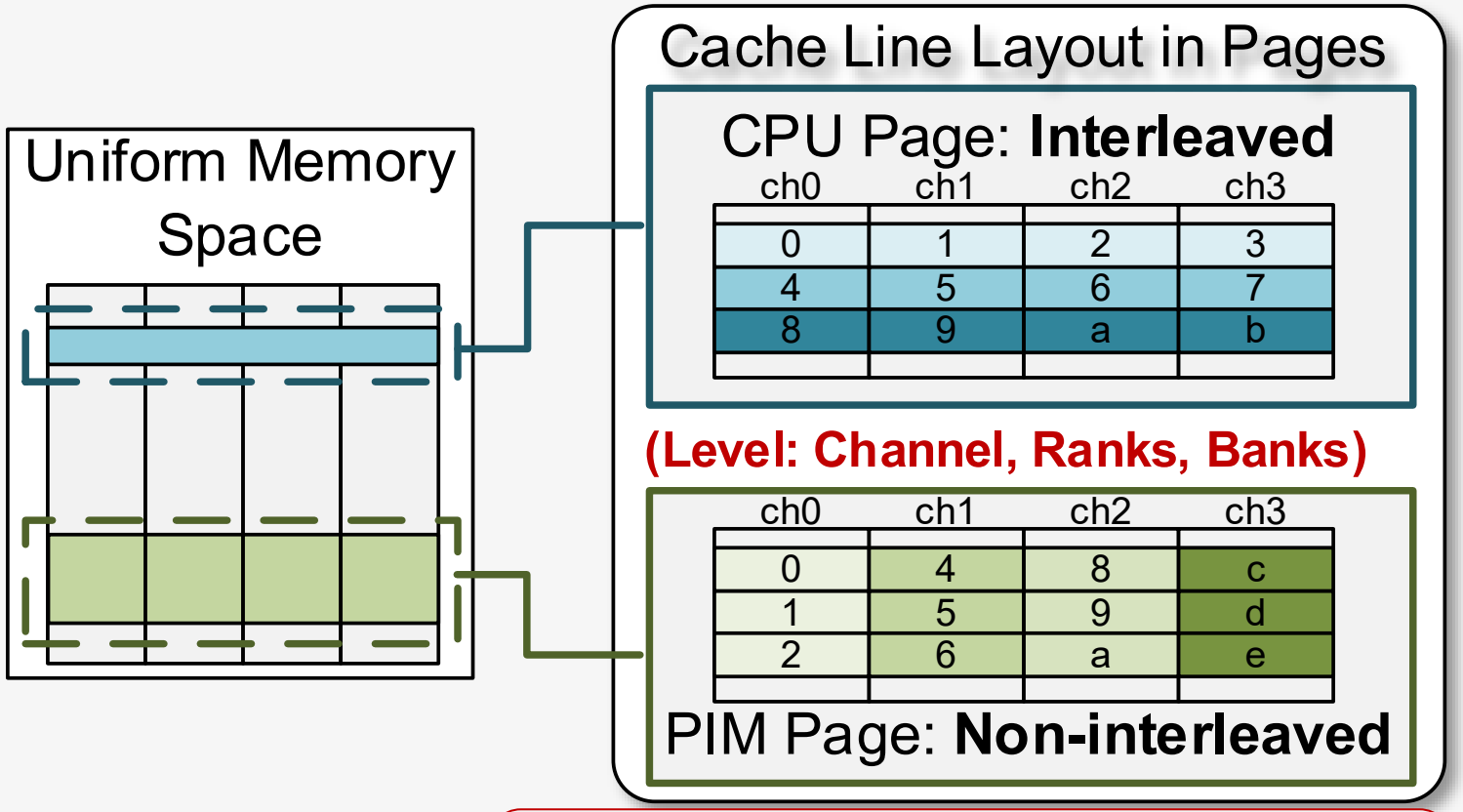



**Next: non-interleaved data layout inside pages**

# UM-PIM Motivation: A Uniform Memory Space

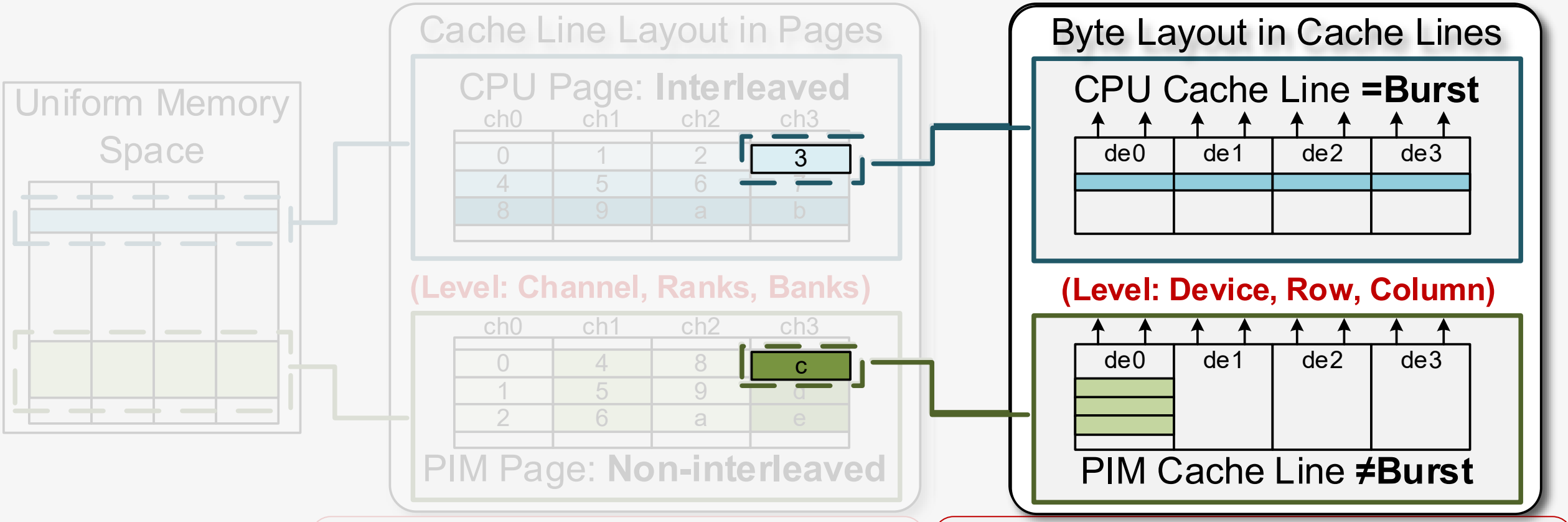
- Challenge 1: co-existence of CPU & PIM's pages under a single VM space
- **Challenge 2: implementing two different data layouts with low cost**




**Addressable:**
**B.1** Dynamic Address Mapping

# UM-PIM Motivation: A Uniform Memory Space

- Challenge 1: co-existence of CPU & PIM's pages under a single VM space
- **Challenge 2: implementing two different data layouts with low cost**

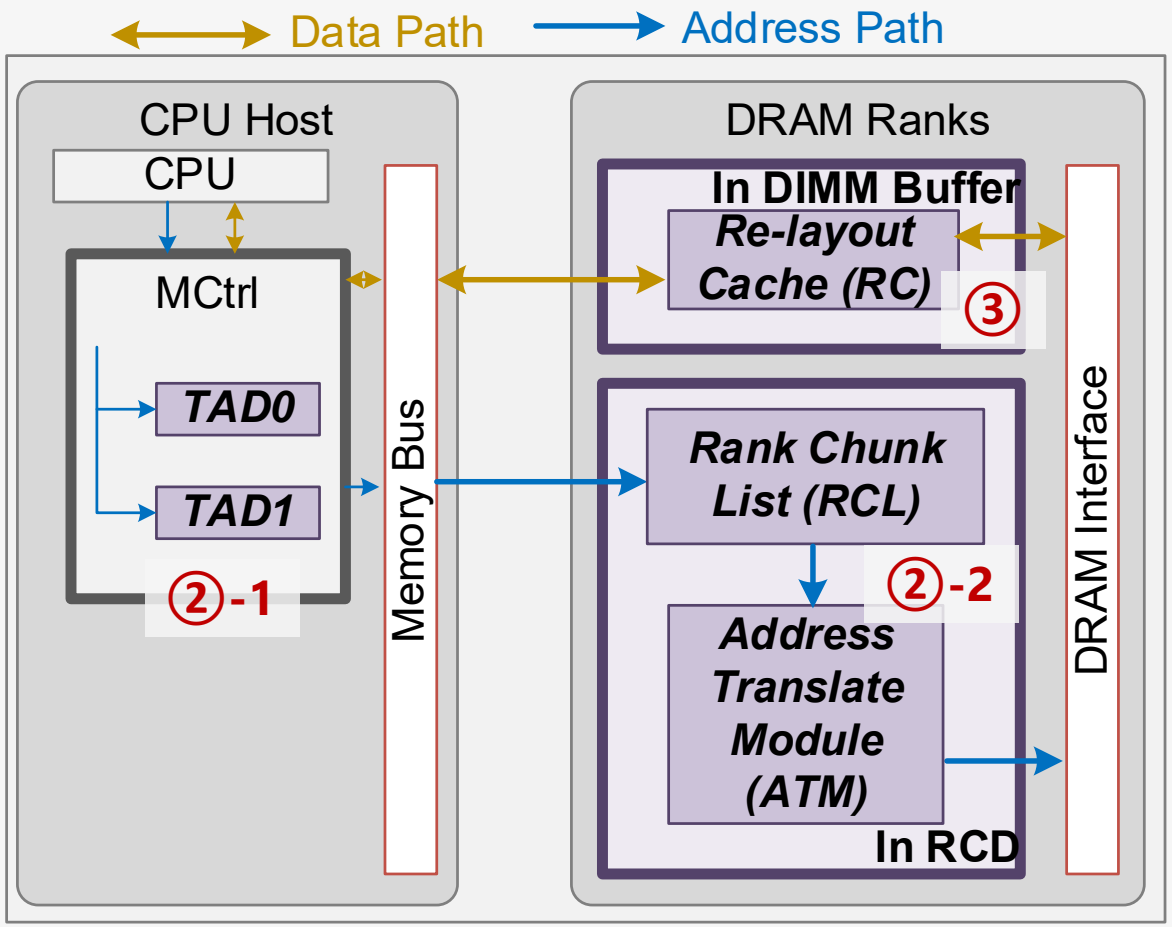



**Addressable:**
**B.1** Dynamic Address Mapping

**Not Addressable:**
**B.2** Hardware for Data Re-layout

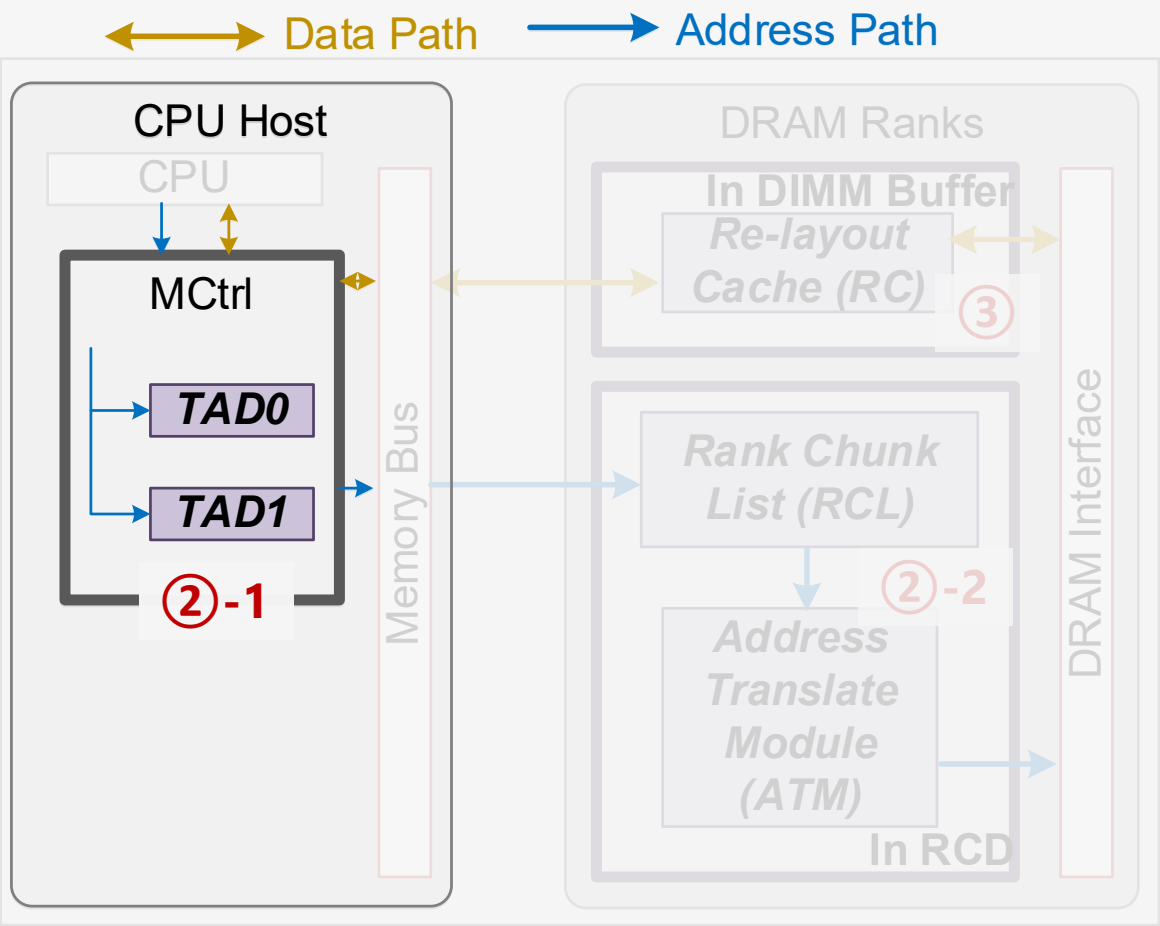
# B. Data Layout -- Overview

- B.1 Cache line layout inside page
  -- Dynamic Address Mapping ②
  - Above Bank (Visible to Memory Controller): Config Memory Controller ②-1
  - Below Bank (Invisible to Memory Controller): DRAM-side Hardware (ATM&RCL) ② -2
- B.2 Byte layout inside Cache Line
  -- Data re-layout ③
  - DRAM-side hardware (RC)



# B. Data Layout -- Overview

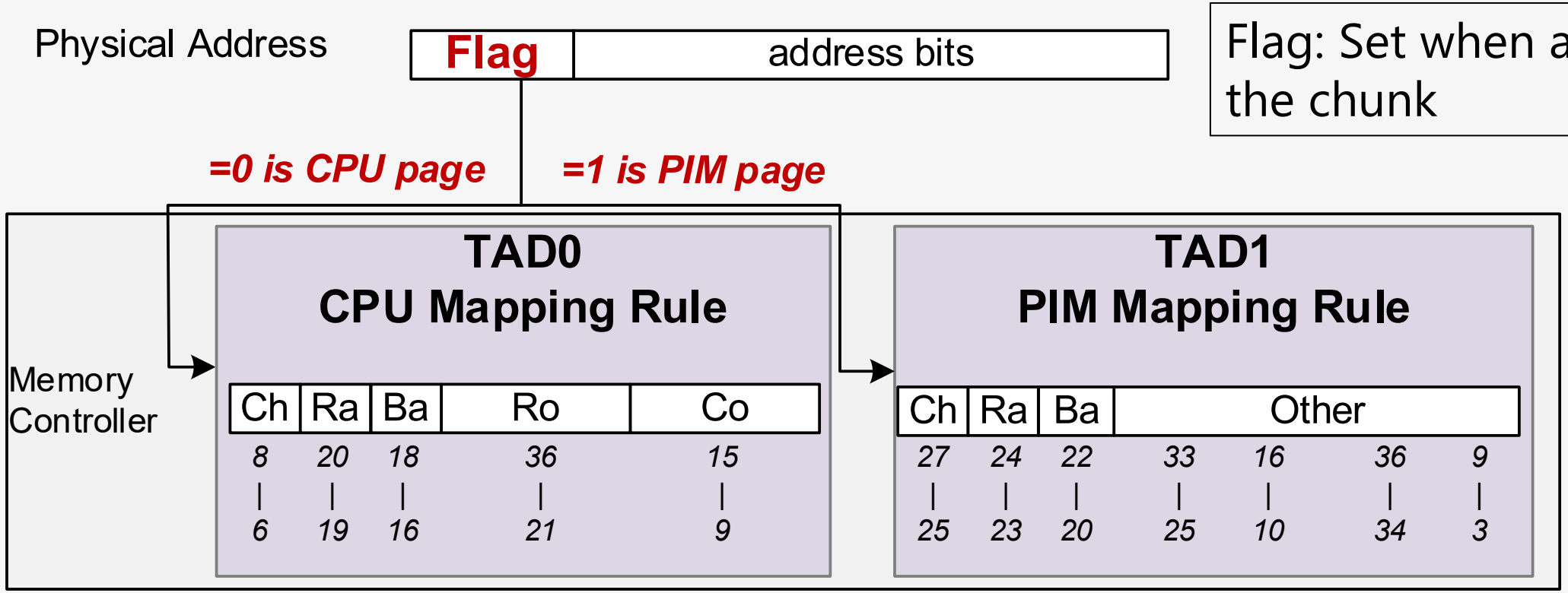
- B.1 Cache line layout inside page
  -- Dynamic Address Mapping ②
  - Above Bank (Visible to Memory Controller): Config Memory Controller ②-1
  - Below Bank (Invisible to Memory Controller): DRAM-side Hardware (ATM&RCL) ② -2
- B.2 Byte layout inside Cache Line
  -- Data re-layout ③
  - DRAM-side hardware (RC)



# B. Data Layout — Address Mapping (Above Bank) ②-1

- **Above Bank:** Add an address alias with Target Address Decoder (TAD) in memory controller, TAD1 have different address mapping from the origin TAD0

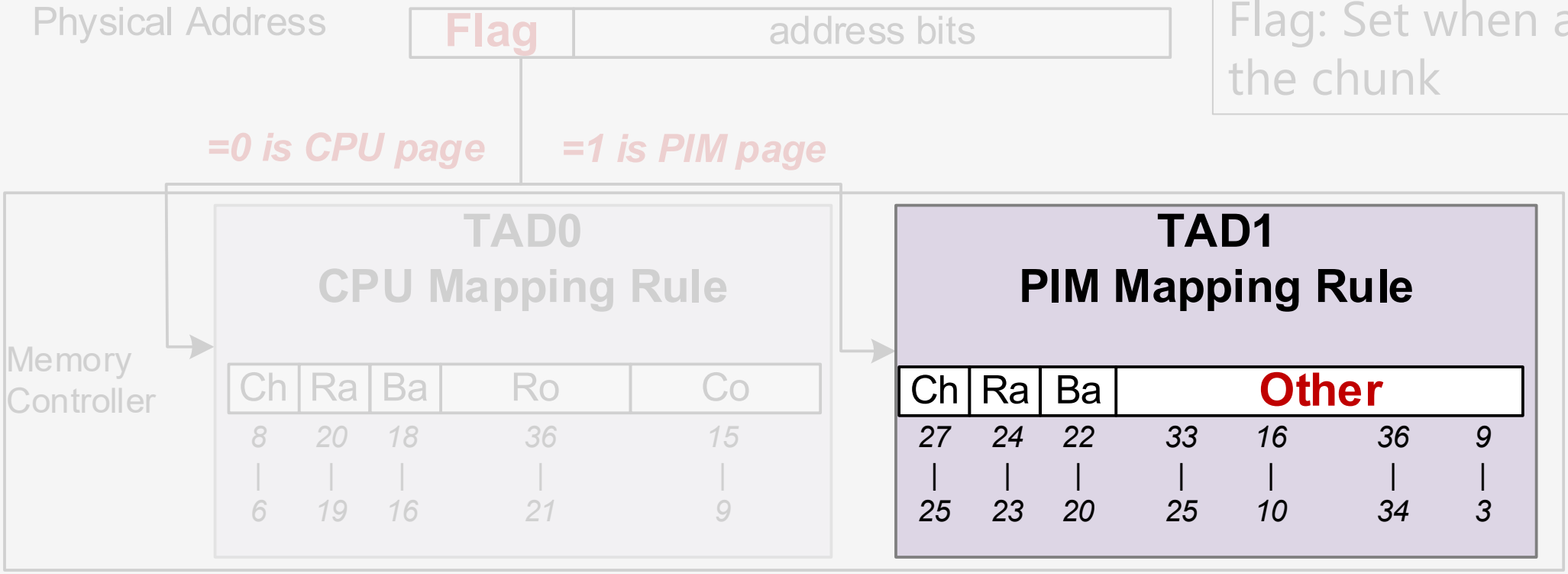
Flag: Set when allocating the chunk



# B. Data Layout — Address Mapping (Above Bank) ②-1

**Above Bank:** Add an address alias with Target Address Decoder (TAD) in memory controller, TAD1 have different address mapping from the origin TAD0

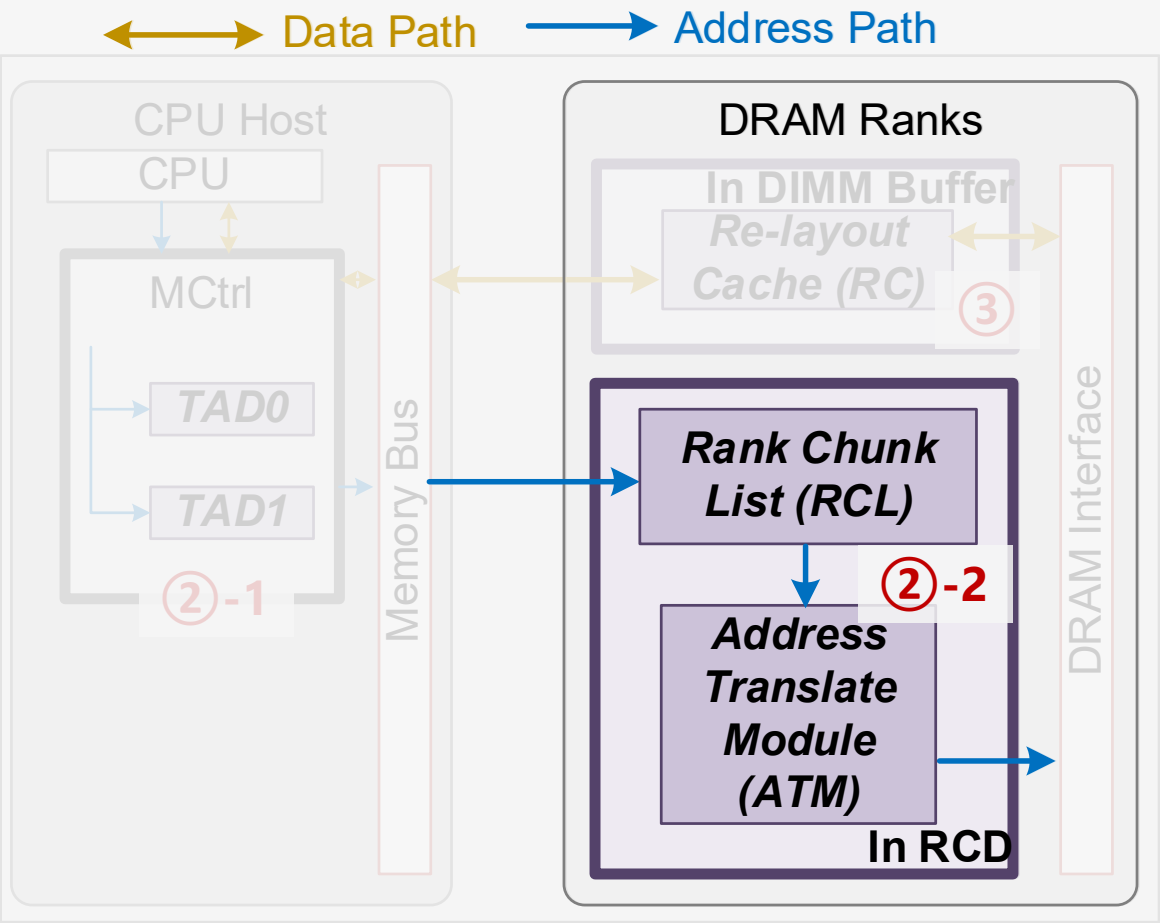
Flag: Set when allocating the chunk




**Below Bank**

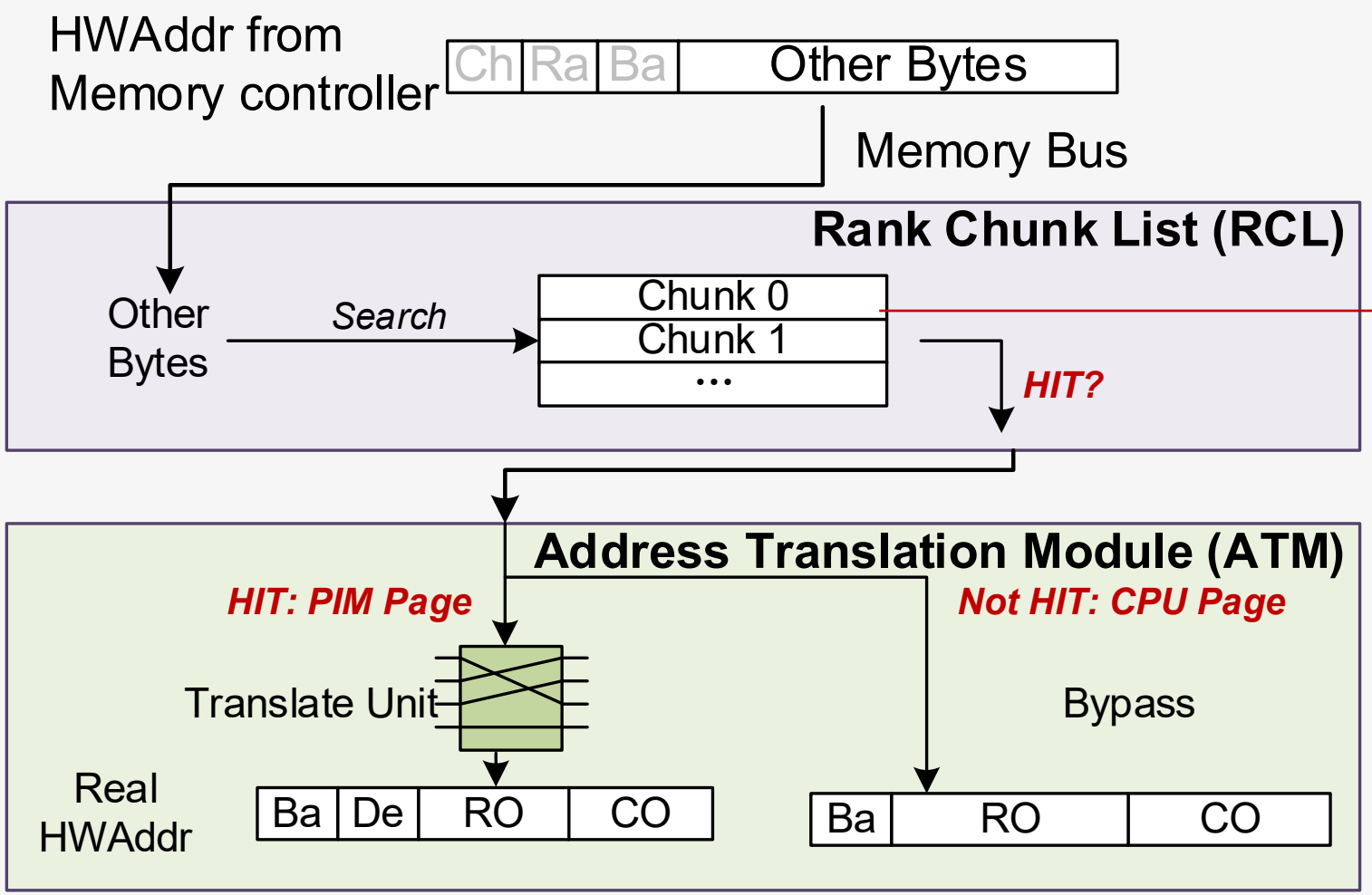
# B. Data Layout -- Overview

- B.1 Cache line layout inside page
  -- Dynamic Address Mapping ②
  - Above Bank (Visible to Memory Controller): Config Memory Controller ②-1
  - Below Bank (Invisible to Memory Controller): DRAM-side Hardware (ATM&RCL) ② -2
- B.2 Byte layout inside Cache Line
  -- Data re-layout ③
  - DRAM-side hardware (RC&CG)



# B. Data Layout — Address Mapping (Below Bank) ②-2

**Below Bank:** 2 hardware modules on DRAM side



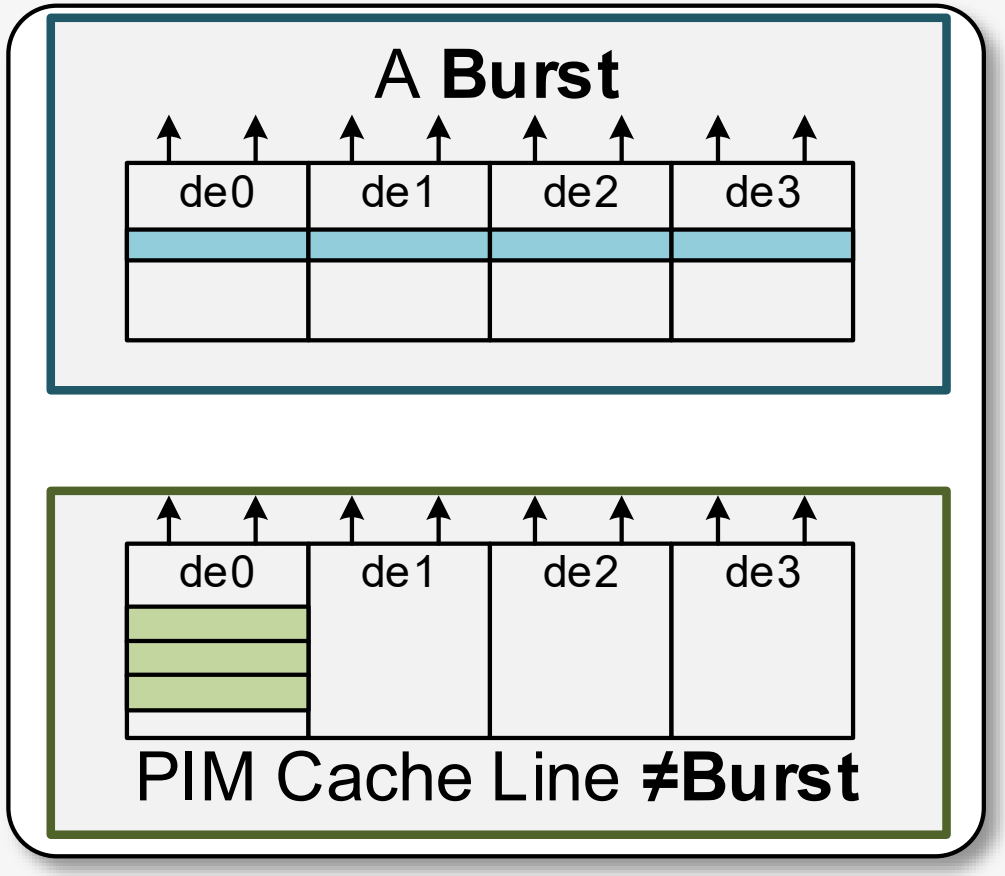

Start addresses of allocated chunks (Set when the chunks are allocated)

**RCL**:
Check whether the address belongs to a PIM page

**ATM**:
If PIM address, remap the address bits to Device-/Column-/Row levels

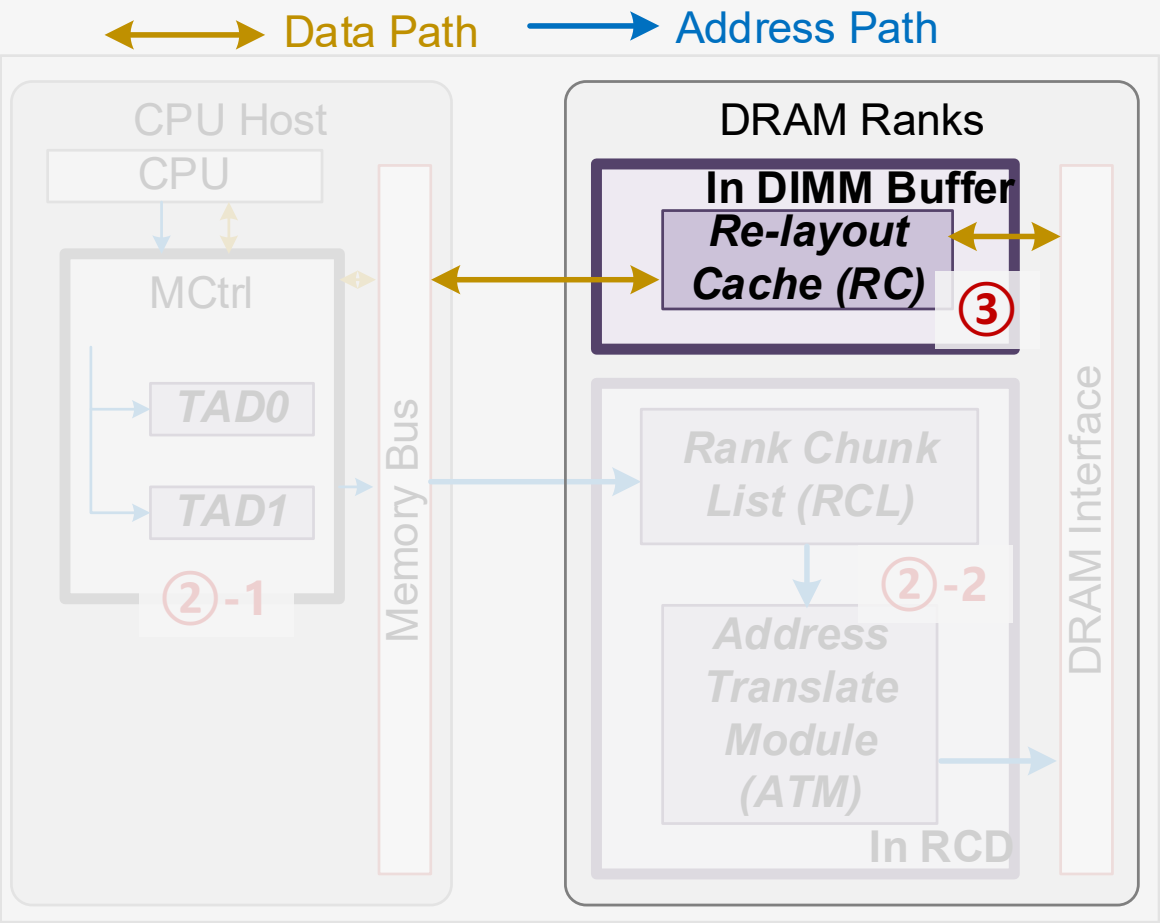
# B. Data Layout — Re-layout ③ Recall the Challenge

**Inside Cache Line:** solve the problem **Cache-Line of PIM Page ≠ Burst**



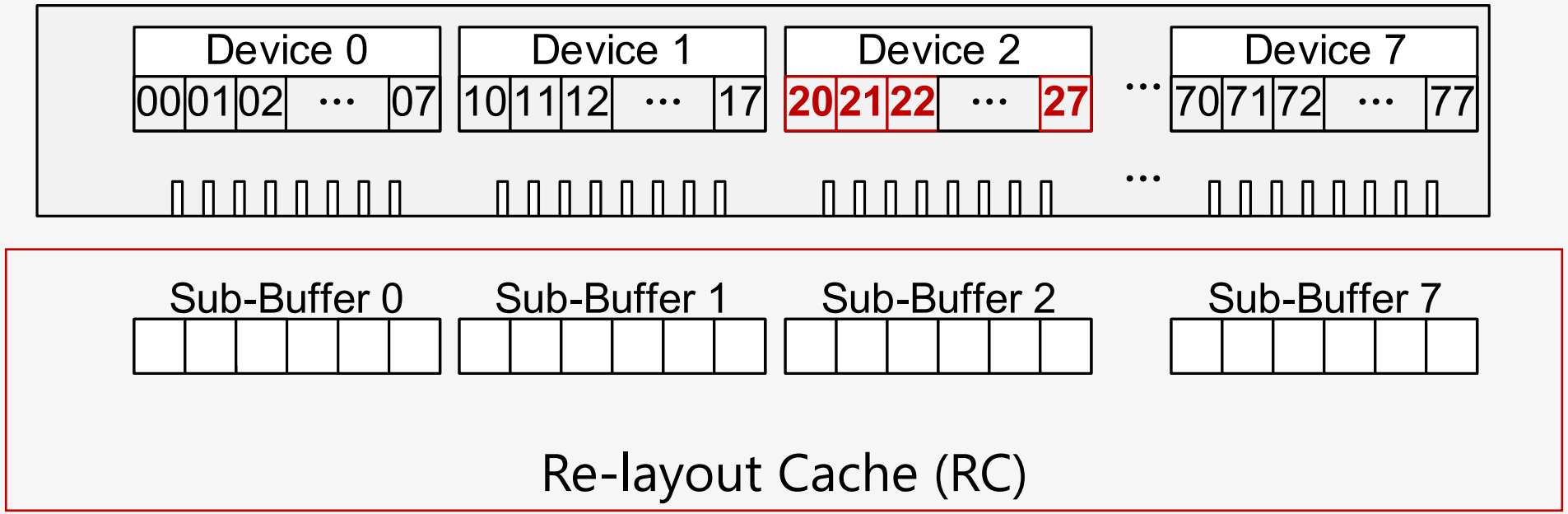
# B. Data Layout -- Overview

- B.1 Cache line layout inside page
  -- Dynamic Address Mapping ②
  - Above Bank (Visible to Memory Controller): Config Memory Controller ②-1
  - Below Bank (Invisible to Memory Controller): DRAM-side Hardware (ATM&RCL) ② -2
- B.2 Byte layout inside Cache Line
  -- Data re-layout ③
  - DRAM-side hardware (RC)



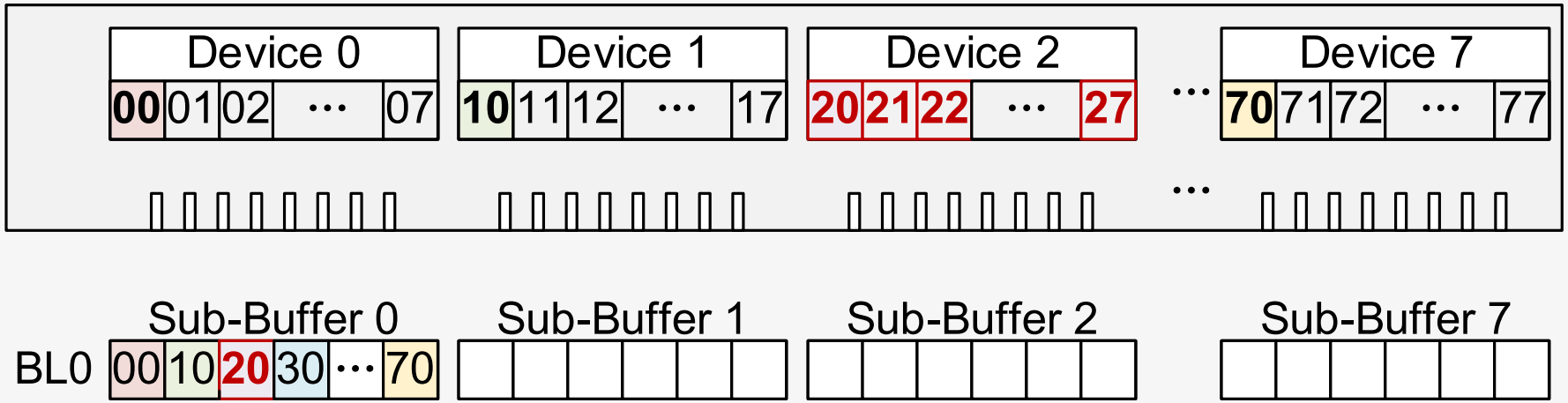
# B. Data Layout — Re-layout ③

- **Ex. of how RC working: Read a 64B cache line of PIM page in Device 2**



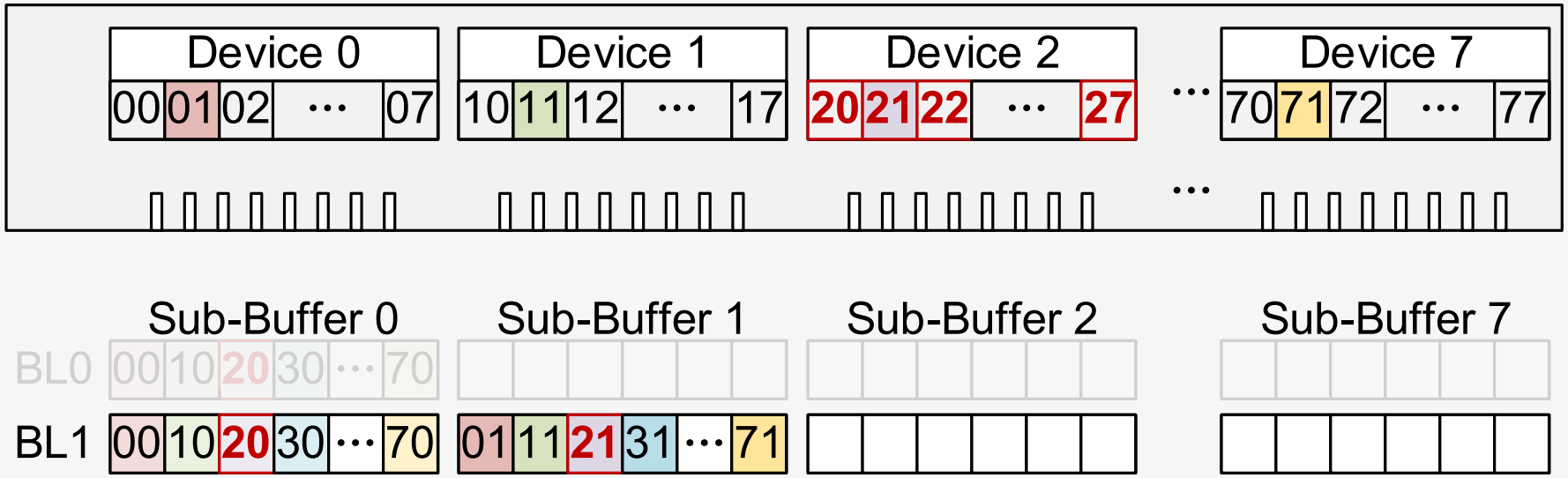
# B. Data Layout — Re-layout ③

- Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2
  - **Cycle 0-7:** Read 8 Burst form every device, cache the Bursts in RC



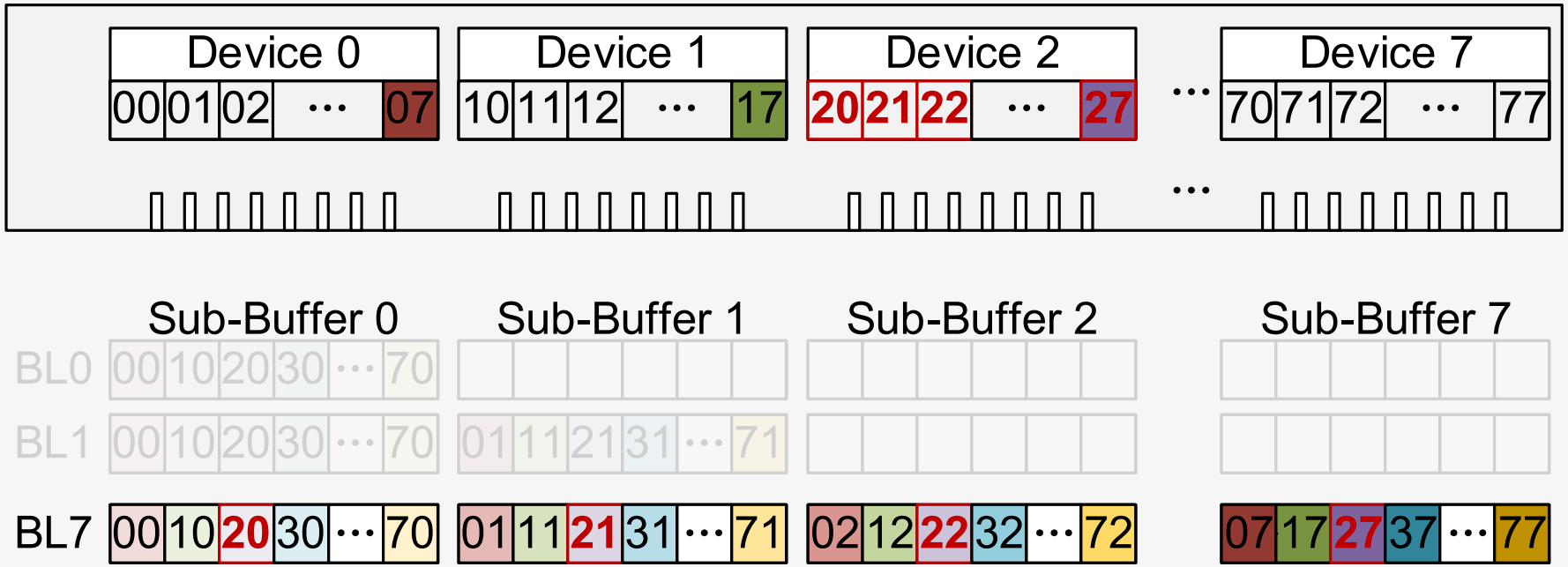
# B. Data Layout — Re-layout ③

- Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2
  - **Cycle 0-7**: Read 8 Burst form every device, cache the Bursts in RC



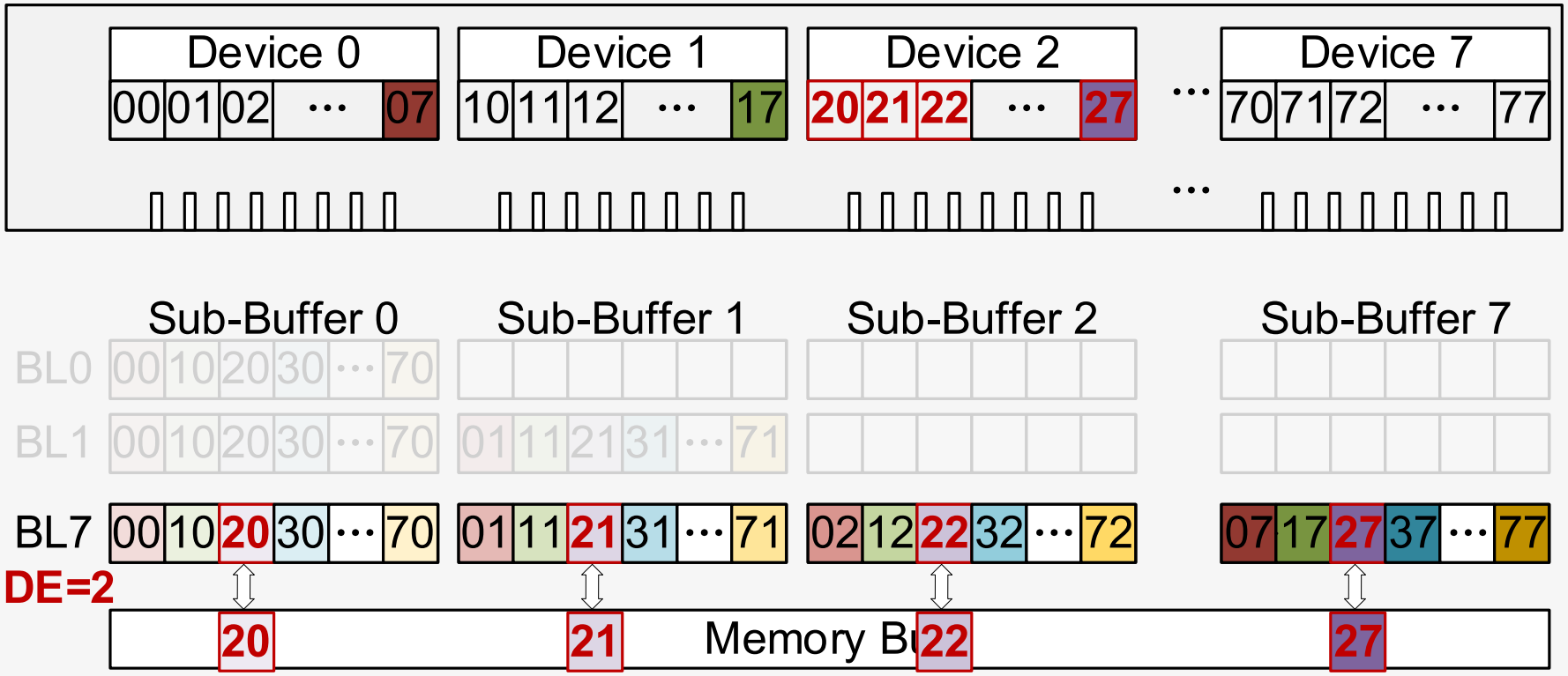
# B. Data Layout — Re-layout ③

- Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2
  - **Cycle 0-7**: Read 8 Burst form every device, cache the Bursts in RC



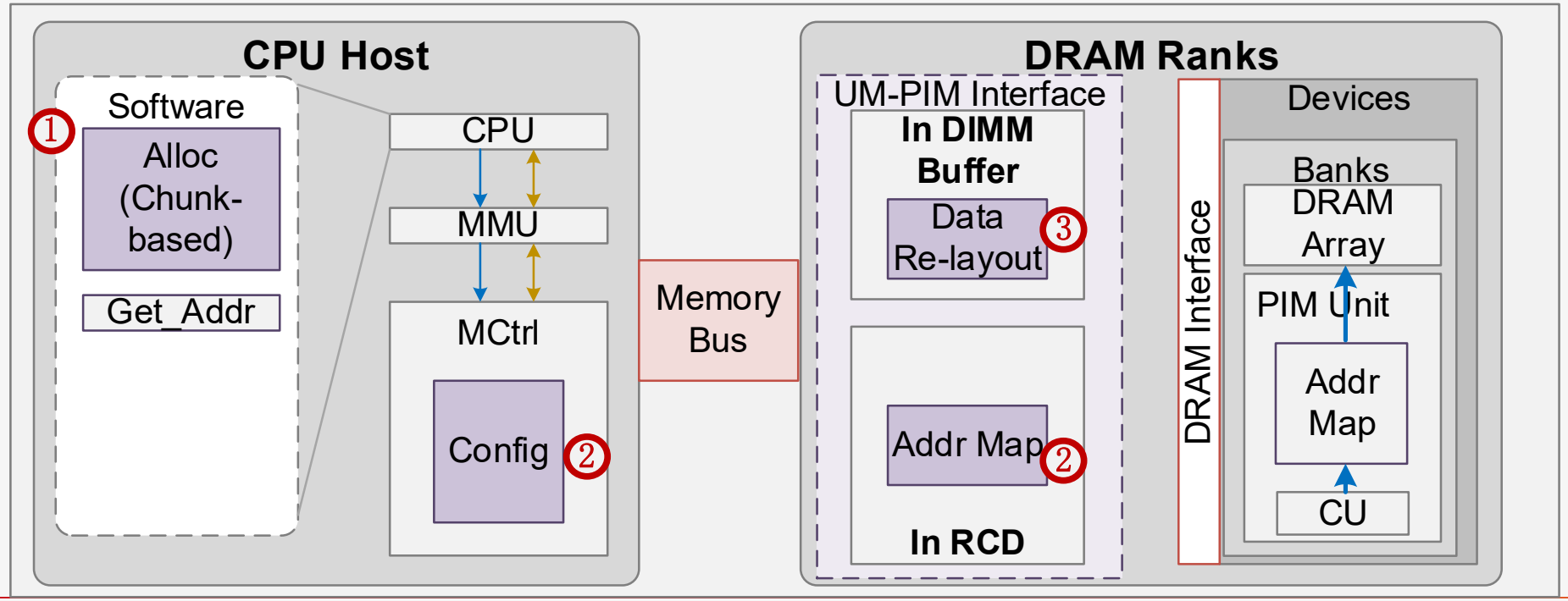
# B. Data Layout — Re-layout ③

- Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2
  - Cycle 0-7: Read 8 Burst form every device, cache the Bursts in RC
  - **Cycle 8**: Get the data from Device 2, form a burst read to CPU



# UM-PIM Summary

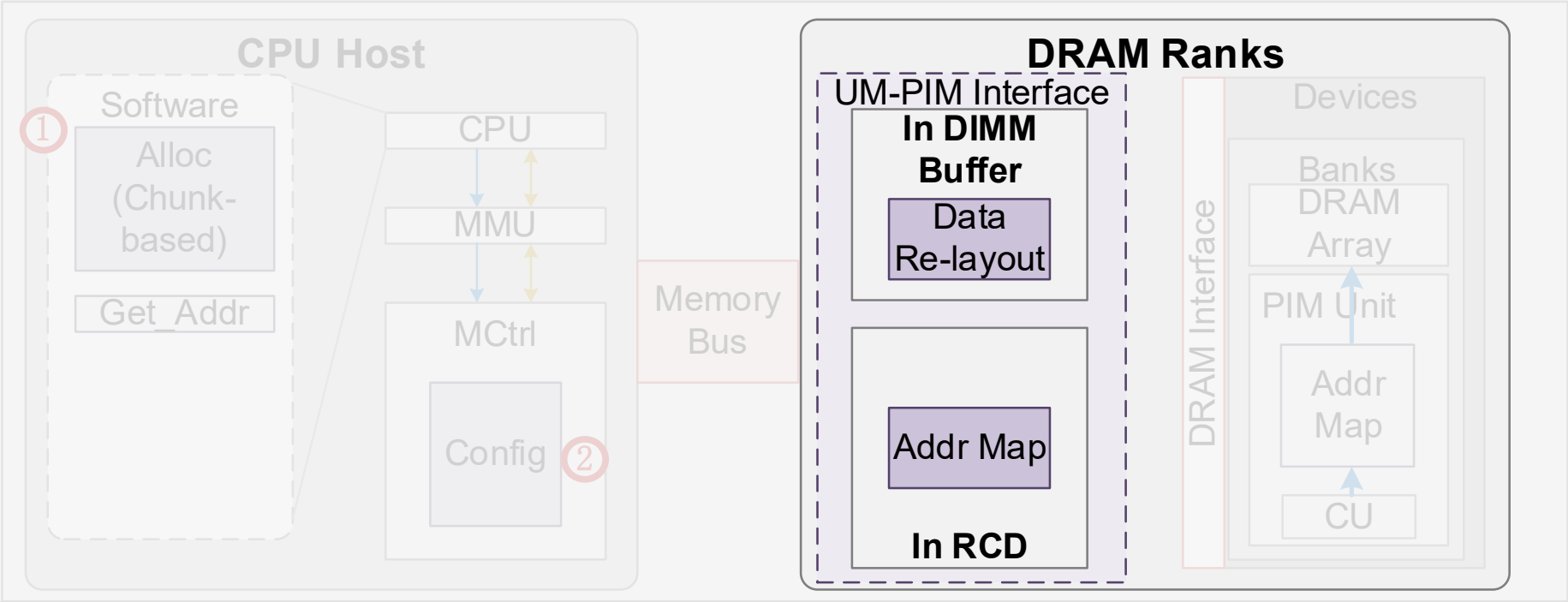
- **A.** Memory Management: ① API for CPU Allocating PIM Pages
- **B.** Data Layout: ② ③ Hardware Modules on DRAM Side



# UM-PIM Summary

- **A.** Memory Management: ① API for CPU Allocating PIM Pages
- **B.** Data Layout: ② ③ Hardware Modules on DRAM Side

**All Hardware modifications are on DRAM side**
**-- Full compatibility to Soc**



# More in the Paper

- Accelerating CPU access PIM pages (SW-HW codesign solution)
- Other details:
  - Detailed description of CPU and PIM accessing PIM pages
  - Extended DRAM instruction support for UM-PIM
  - Inter-PIM-units communication APIs

**UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space**
in ISCA'24

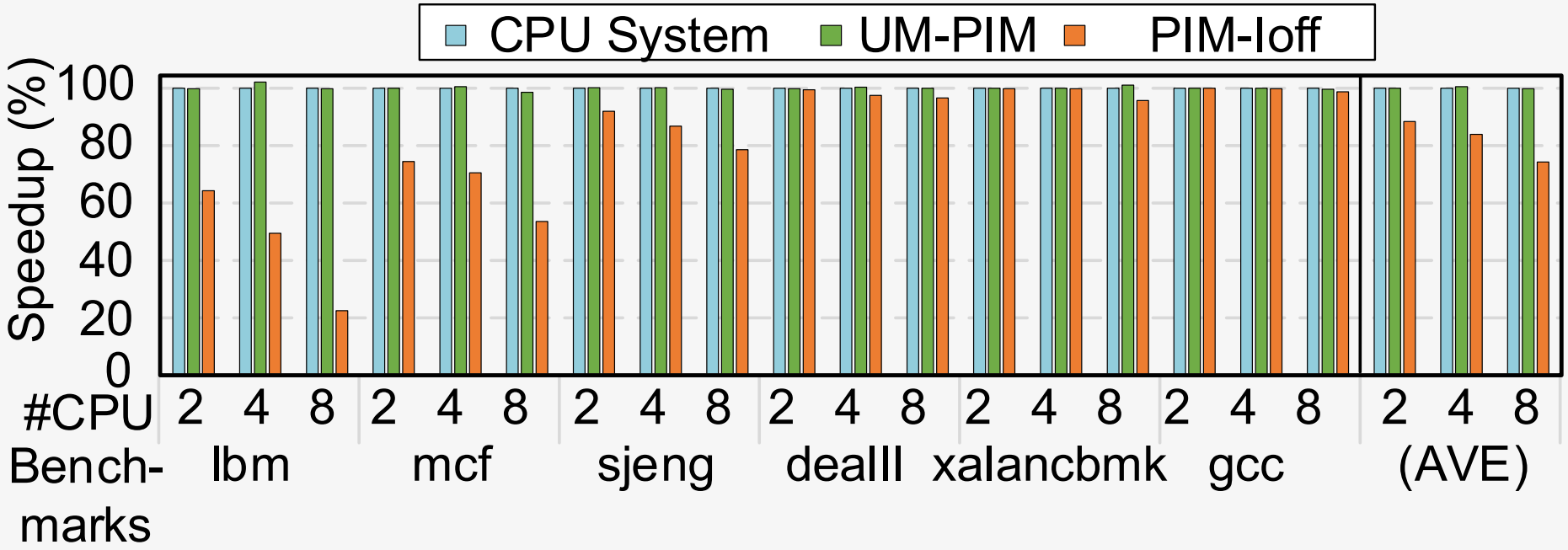
**Yilong Zhao**, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*
*Shanghai Jiao Tong University, Tsinghua University and Huawei*

# Evaluation Methodology

- Simulator & Configuration
  - CPU: GEM5 + Ramulator2;  PIM units: **UPMEM DPU @ 500MHz**
  - DDR4-2400, 8(Channel)x 8(Ranks)
- Benchmarks:
  - CPU Workloads: from **SPEC CPU 2006**
    - lbm, mcf, sjeng, dealII, xalancbmk, gcc
  - PIM Workloads (requires CPU & PIM): from **PRIM**
    - BFS, PR, MLP, UNI, TC, SCAN-RSS/SSA, SEL, HST, NW, WFA, RL
- Baseline:
  - PIM-Ion: PIM System with **Memory interleaving switched on**
  - PIM-Ioff: PIM System with **Rank & Channel interleaving switched off**, (e.g. UPMEM)

# Results on CPU Workloads

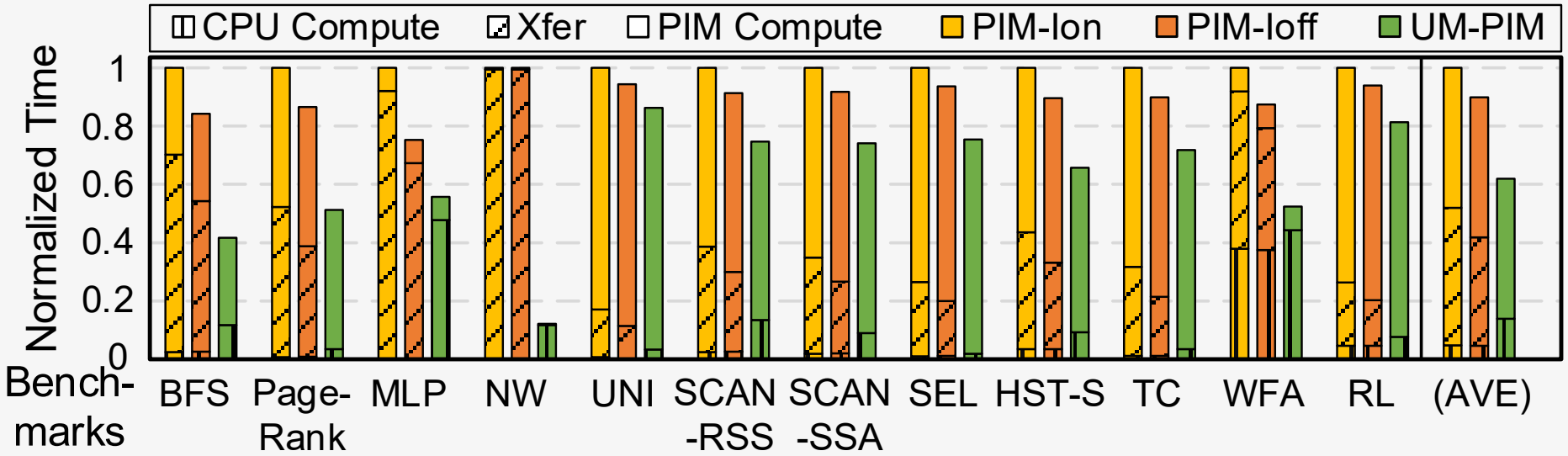
- UM-PIM does not switch off the interleaving




- PIM-Ioff: **22%** performance loss in average, interleaving switched off
- More CPUs in the system, larger perf. loss
- UM-PIM: **<0.1%** performance loss

# Results on PIM Workloads

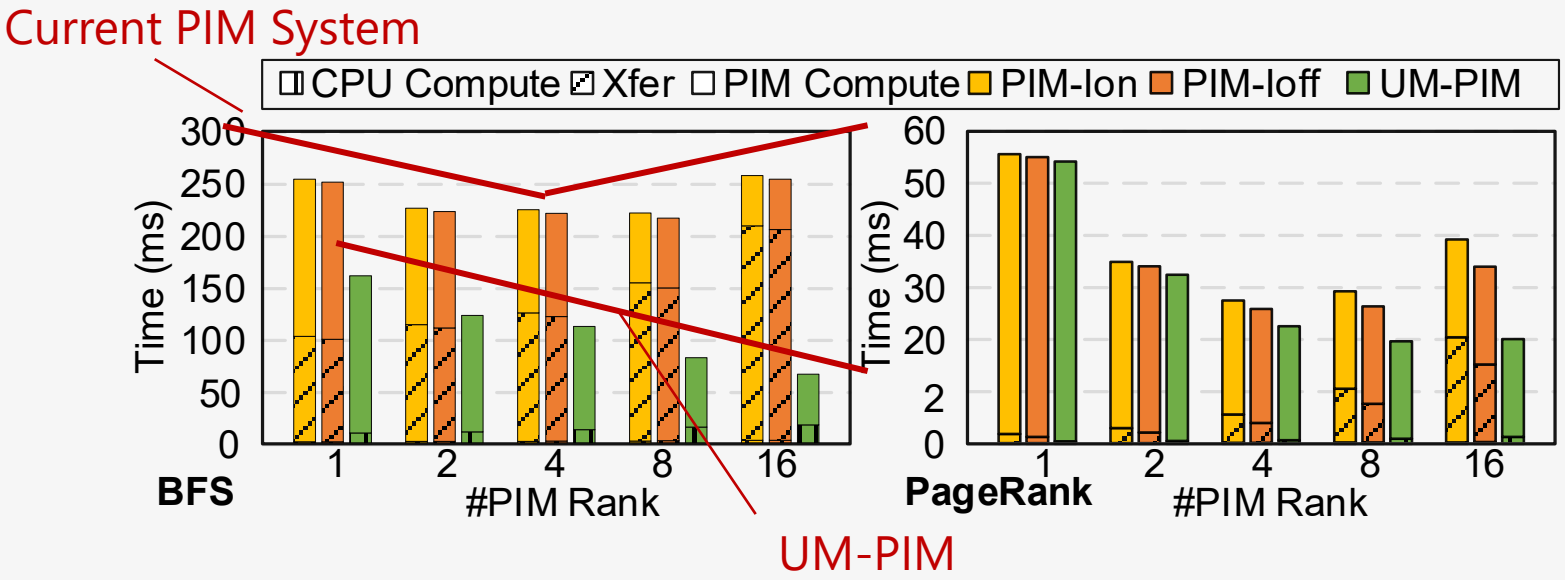
- UM-PIM has no data transfer between isolated memory spaces




- Compared to PIM-Ioff (Rank & Channel interleaving switched off, UPMEM)
- UM-PIM has **4.93×** reduction on CPU time (including Computing and Data Transfer), resulting in **1.96×** speed up in average on the **whole benchmark**.

# Results on PIM Workloads

- UM-PIM has no data transfer between isolated memory spaces




- For workloads with intensive inter-PIM-units data transfer, UM-PIM can still benefit from computing using more PIM units
- In contrast, time on current PIM systems increases when using more PIM units

# Summary

Key advantages of UM-PIM:

- **Uniform & shared memory space:** CPU and PIM pages co-exist in a uniform memory space, and data transfer is eliminated compared to isolated space design
- **Fast & Transparent data layout:** Dynamic address mapping and data re-layout for two kinds of pages are processed fast and transparently by DRAM-side hardware module
- **Compatibility with current CPU systems:** no modifications on CPU-side hardware and CPU page's memory management.

## UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

in ISCA'24

**Yilong Zhao**, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University and Huawei*

# PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

in ASPLOS'25

**Yilong Zhao**, Mingyu Gao, Huanchen Zhang, Fangxin Liu,* Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University*

Wednesday, **March 25 8:30 – 10:35 AM**
**OHIO & MONOGAHELA**, Session 4D

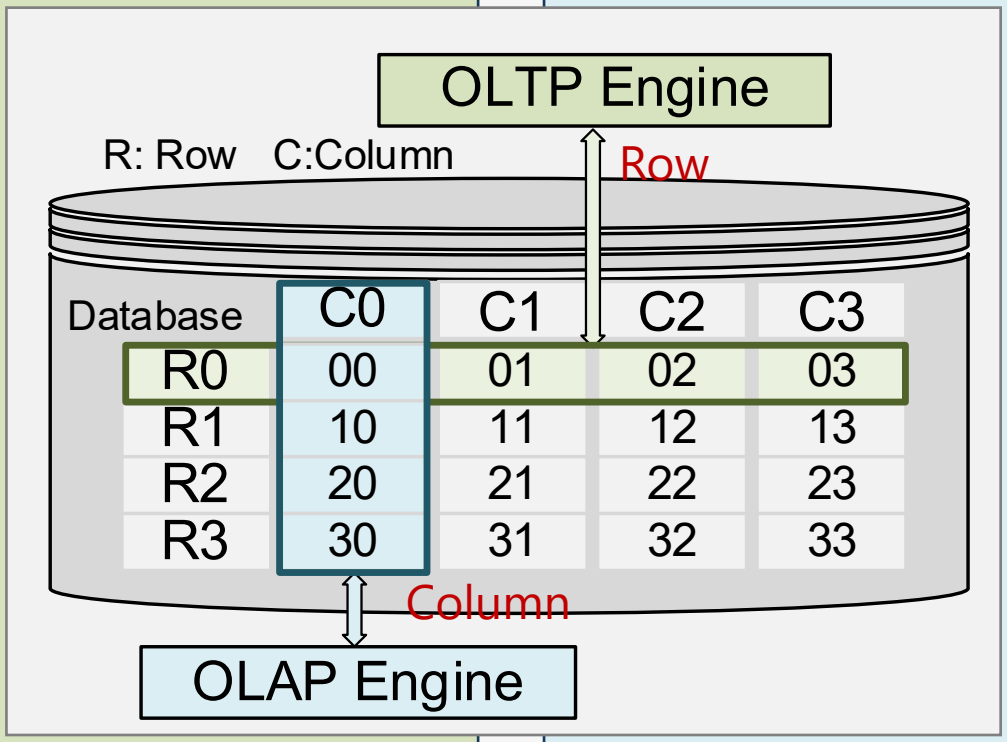
# HTAP: The Convergence of Conflicting Workloads

Hybrid transaction/analytical processing (HTAP) database, 2 processing sets:

**Online transaction processing (OLTP):**

**Access Pattern:**
Row-wise, random
**Bottleneck**:
Computation-intensive

**Ideal Format:**
**Row-store**




**Online analytical processing (OLAP):**

**Access Pattern:**
Column-wise, sequential
**Bottleneck**:
Memory-intensive

**Ideal Format:**
**Column-store**

# HTAP: The Convergence of Conflicting Workloads

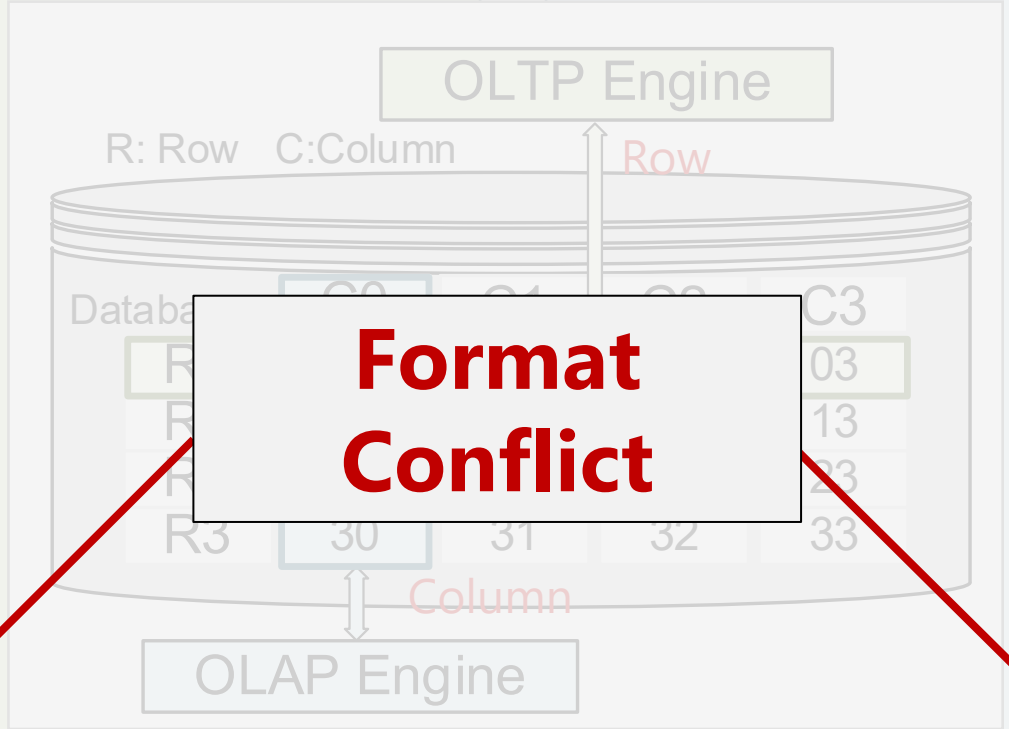
Hybrid transaction/analytical processing (HTAP) database, 2 processing sets:

**Online transaction processing (OLTP):**

**Access Pattern:**
Row-wise, random
**Bottleneck**:
Computation-intensive

**Online analytical processing (OLAP):**

**Access Pattern:**
Column-wise, sequential
**Bottleneck**:
Memory-intensive




**Format Conflict**

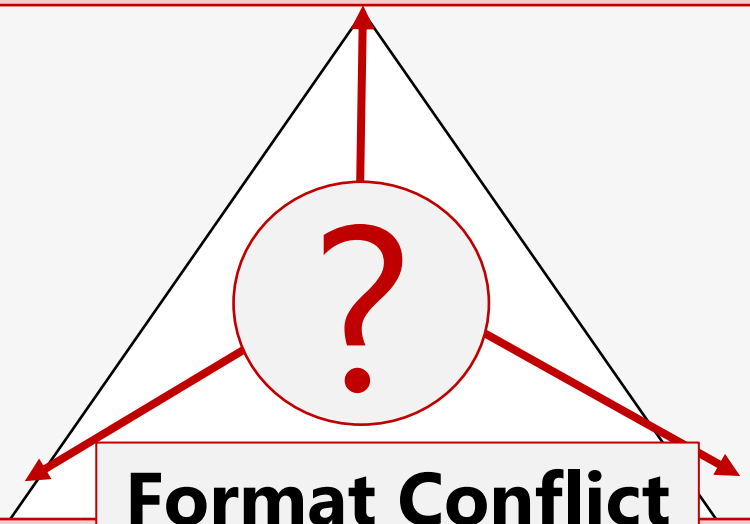
**Ideal Format: Row-store**

**Ideal Format: Column-store**

# The "Impossible Trinity" of HTAP Design Goals

Three goals of an ideal HTAP:[SIGMOD'17]

**Workload-specific optimizations (*W*):**
Optimized performance for both workloads




**Format Conflict**

**Performance isolation (*I*):**
Zero performance interference

**Data freshness (*F*):**
OLAP need up-to-date data

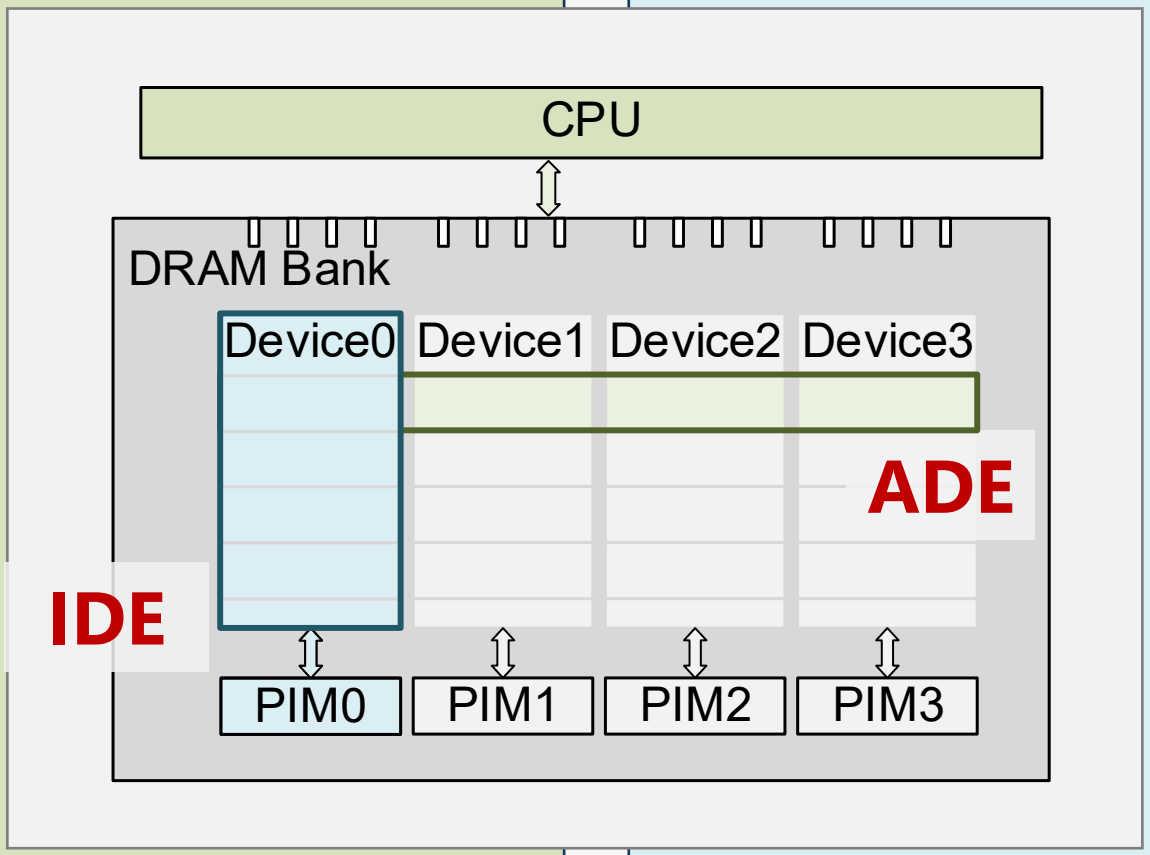
**Current solutions sacrifice one or more goals to achieve the others**

# Access Divergence in CPU-PIM System

## CPU

Interleaved Access for high bandwidth

→ **Across devices (ADE) Access**




## PIM

Contiguous access because PIM only visible to local device

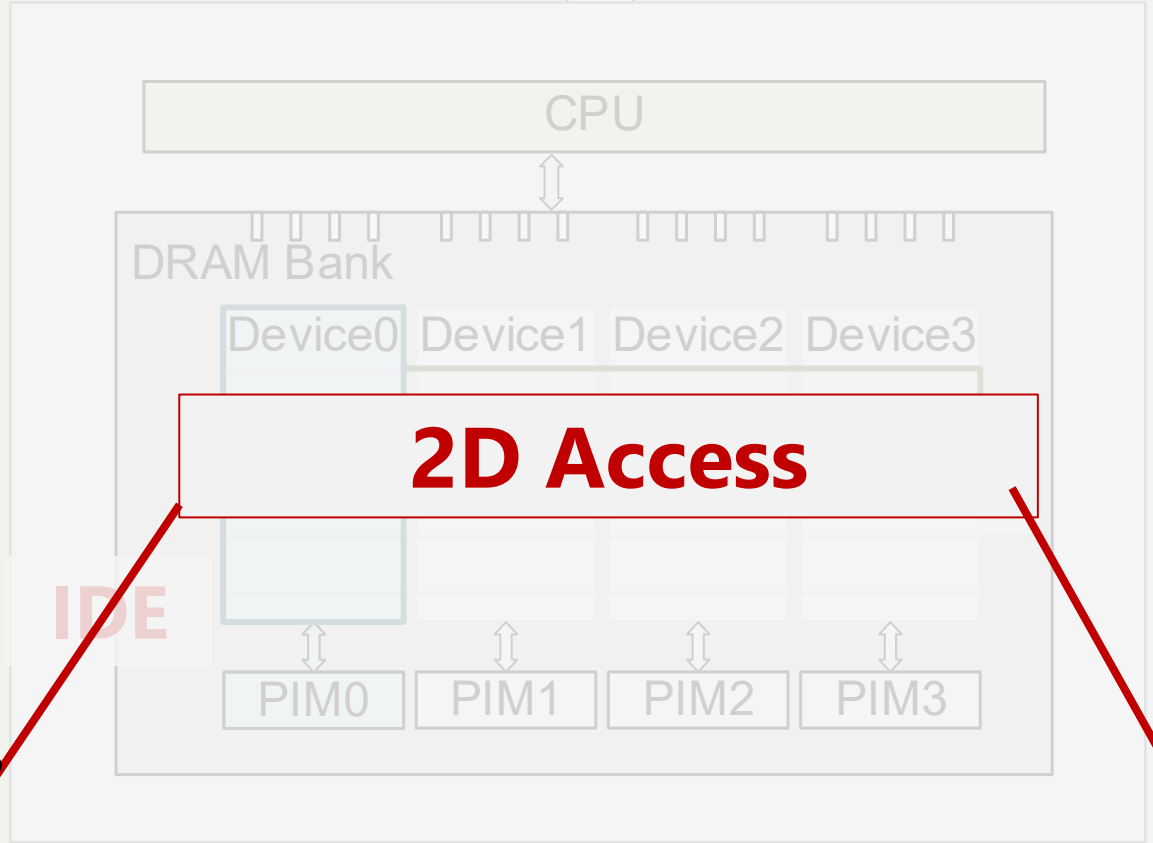
→ **Inside device (IDE) Access**

# Access Divergence in CPU-PIM System

## CPU

Interleaved Access for high bandwidth

→ Across devices (ADE) Access




**2D Access**

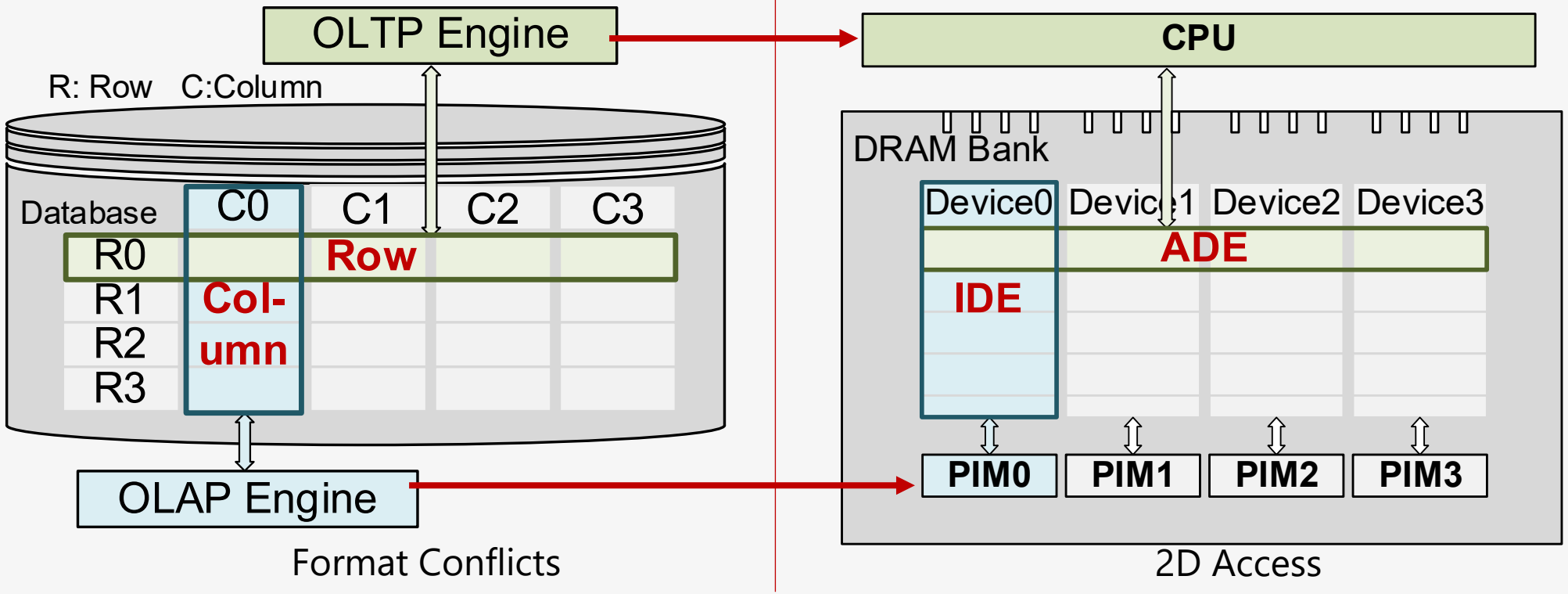
## PIM

Contiguous access because PIM only visible to local device

→ Inside device (IDE) Access

# Key Insight: Format Conflict vs. Access Divergence

| HTAP | | | | | PIM |
|---|---|---|---|---|---|
| OLTP | Row-wise | → | Across Device (ADE) | CPU | |
| OLAP | Column-wise | → | Inside Device (IDE) | PIM | |



# The PUSHtap Vision: Hardware-Software Co-alignment

## Objective: Single Instance, Unified Format

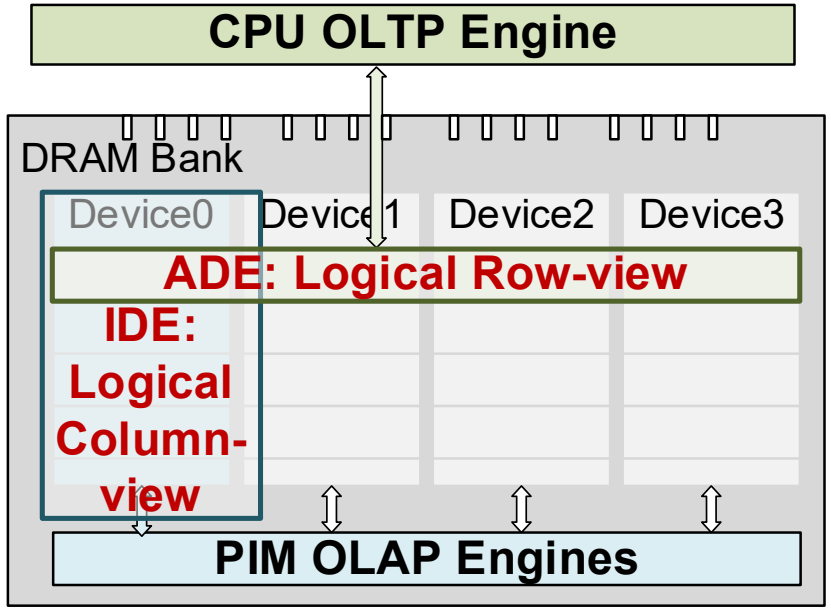
-- Satisfies both row & column operations, simultaneously achieve 3 goals

- *Workload-specific Optimization:* *Unified format -> both row & column access*
- *Performance Isolation:* *No consistency (merge/rebuild) overhead*
- *Data Freshness:* *Processed on the same instance*

Core Approach: **Unified Data Format**

Logical Rows: **Aligned** to ADE

Logical Columns: **Aligned** to IDE



# Challenges

- **Challenges → Solutions:**

| | | |
|---|---|---|
| 1 Data Alignment Challenge Due to Heterogeneous Data Types | → | Compact Aligned Format |
| 2 Fragmentation in Version Control | → | Format-aware MVCC & Defragmentation |
| 3 CPU-PIM Contention | → | Architecture support: Two-phase Execution |

**PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format**

in ASPLOS'25

**Yilong Zhao**, Mingyu Gao, Huanchen Zhang, Fangxin Liu,* Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

*Shanghai Jiao Tong University, Tsinghua University*

Wednesday, **March 25 8:30 – 10:35 AM**
**OHIO & MONOGAHELA**, Session 4D

上海交通大學

SHANGHAI JIAO TONG UNIVERSITY

# Thank you!

饮 水 思 源　爱 国 荣 校